

CO1

- 1) Process Framework Activities (CPMCD)
- 2) Umbrella Activities
- 3) Software Myths & Realities
- 4) Incremental and Spiral Models

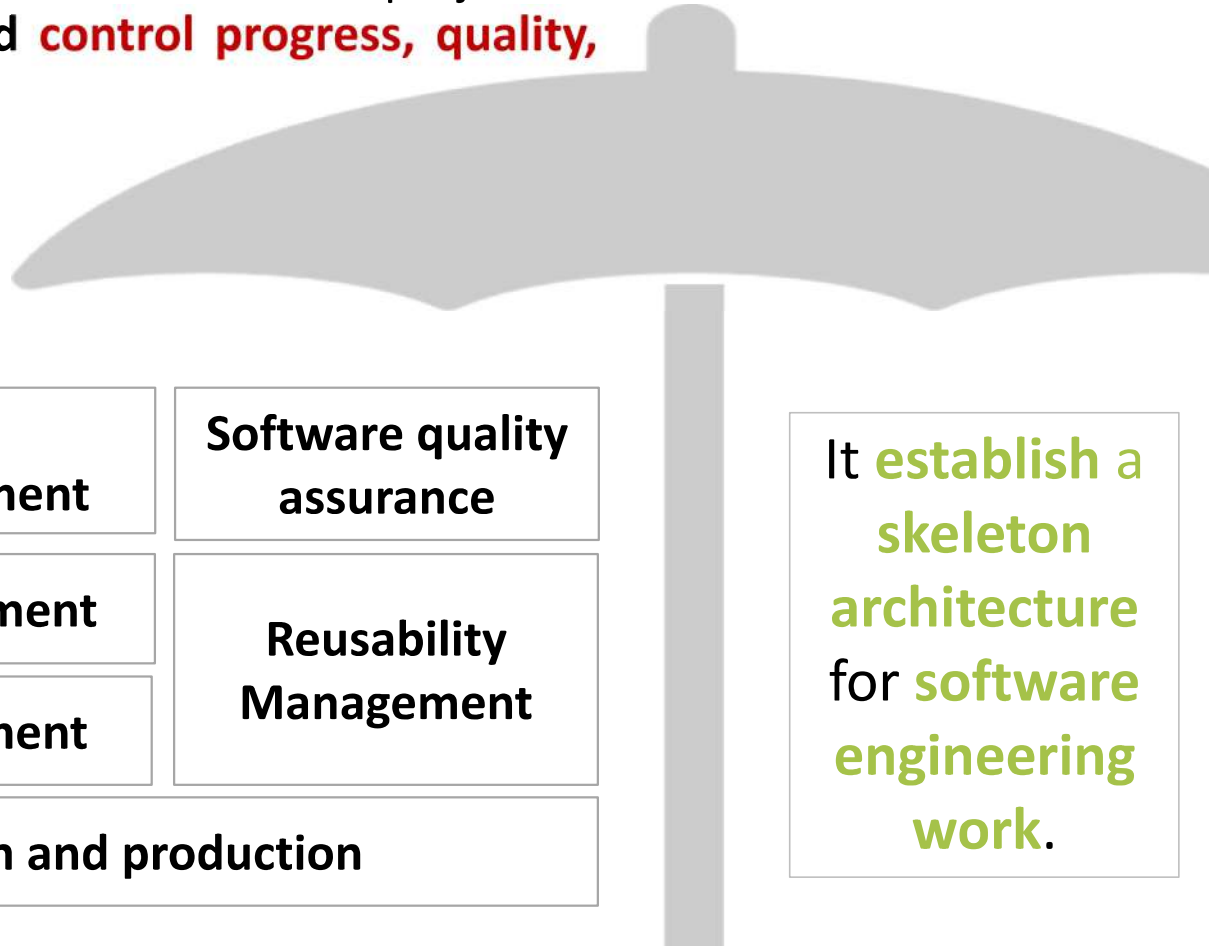
Process Framework Activities (CPMCD)

A process framework establishes the foundation for complete software engineering process, it encompasses five activities

Communication		Communication with Customers / stockholders to understand project requirements for defining software features	Planning		Software Project Plan which defines workflow that is to follow. It describes technical task, risks, resources, product to be produced & work schedule
Modeling		Creating models to understand requirements and shows design of software to achieve requirements	Construction		Code Generation (manual or automated) & Testing (to uncover errors in the code)
Deployment		Deliver Software to Customer Collect feedback from customer based on evaluation Software Support			

Umbrella Activities

- ▶ **Umbrella activities** applied throughout the software project & help a software team to manage and **control progress, quality, change & risks**
- ▶ Umbrella activities are those which keep running in the background throughout the software development



**Software project
Tracking & Control**

**Risk
Management**

**Software quality
assurance**

Technical Reviews

Measurement

**Reusability
Management**

Software Configuration Management

Work product preparation and production

It **establish a
skeleton
architecture
for software
engineering
work.**

Umbrella Activities Cont.

- ▶ **Software project tracking and control:** allows the software team to **assess progress against the project plan** and take any necessary action to maintain the schedule.
- ▶ **Risk management:** assesses (**evaluates**) **risks** that may affect the outcome of the project or the quality of the product.
- ▶ **Software quality assurance:** defines and conducts the activities required to **ensure software quality**.
- ▶ **Technical reviews:** **assesses** software engineering **work** products in an effort **to uncover** and remove **errors** before they are propagated to the next activity.
- ▶ **Measurement:** defines and collects process, project and product measures that assist the team in delivering software that meets stakeholders' needs.
- ▶ **Software configuration management:** it manages the effects of change throughout the software process.

Umbrella Activities Cont.

- ▶ **Reusability management:** it defines criteria for work product reuse (including software components) and establishes **mechanisms to achieve reusable components**.
- ▶ **Work product preparation and production:** it encompasses (includes) the activities required to create work products such as **models, documents, logs, forms and lists**.

Software Myths

Beliefs about software and the process used to build it.



“Misleading Attitudes that cause serious problem” are myths.



Management Myths



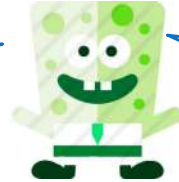
Customer Myths



Practitioner's
(Developer)
Myths

Management myth - 1 & 2

We **have standards and procedures** to build a system, which is enough.



We have **the newest computers and development tools.**

Reality

- ▶ Are software **practitioners** aware of standard's existence?
- ▶ Does it **reflect modern software engineering** practice?
- ▶ Is it **complete**?
- ▶ Is it streamlined to **improve time to delivery** while **still maintaining a focus on quality**?
- ▶ In many cases, the answer to all of these questions is "no."

Reality

- ▶ It **takes much more than the latest model** computers to do high-quality software development.
- ▶ **Computer-aided software engineering (CASE)** tools are more important than hardware.

Management myth - 3 & 4

We **can add more programmers** and can catch up the schedule.



I **outsourced the development** activity, now I **can relax** and **can wait** for the **final** working **product**.

Reality

- ▶ Software **development is not a mechanistic process** like manufacturing.
- ▶ In the words of Fred Brooks : "**adding people to a late software project makes it later.**"
- ▶ **People** who were **working** must **spend time educating** the newcomers.
- ▶ People can be added but only **in a planned and well-coordinated** manner.

Reality

- ▶ If an **organization** does **not understand how to manage** and **control** software projects internally, it will invariably struggle when it outsources software projects.

Customer myth - 1 & 2

A **general statement of objectives** (requirements) is **sufficient** to start a development.



Requirement Changes can be **easily accommodated** because software is very flexible.

Reality

- ▶ Comprehensive (**detailed**) **statements** of requirements is not always possible, an **ambiguous** (unclear) “**statement of objectives**” can lead to disaster.
- ▶ Unambiguous (clear) requirements can be gathered only through effective and continuous communication between customer and developer.

Reality

- ▶ It is true that software **requirements change**, but the **impact** of change **varies with the time** at which it is introduced.
- ▶ When requirements changes are requested early the cost impact is relatively small.

Practitioner's (Developer) myth – 1 & 2

Once we **write** the **program**, our **job** is **done**.



I **can't** access **quality** until it is **running**.

Reality

- ▶ Experts say "**the sooner you begin 'writing code', the longer it will take you to get done.**"
- ▶ Industry data indicates that 60 to 80 % effort expended on software will be after it is delivered to the customer for the first time.

Reality

- ▶ One of the most effective software **quality assurance mechanisms** can be **applied from the beginning** of a project - **the technical review**.
- ▶ Software reviews are more effective "quality filter" than testing for finding software defects.

Practitioner's (Developer) myth – 3 & 4

Working **program** is the **only deliverable** work **product**.



Software engineering is about **unnecessary** documentation.

Reality

- ▶ A **working program** is only one **part of a software configuration**.
- ▶ A variety of work products (e.g., **models, documents, plans**) provide a foundation for successful engineering and, more important, guidance for software support.

Reality

- ▶ Software engineering is not about creating documents. It is about **creating a quality product**.
- ▶ Better quality leads to reduced rework. And reduced rework results in faster delivery times.

Different Process Models

► Process model is selected based on different parameters

- Type of the project & people
- Complexity of the project
- Size of team
- Expertise of people in team
- Working environment of team
- Software delivery deadline



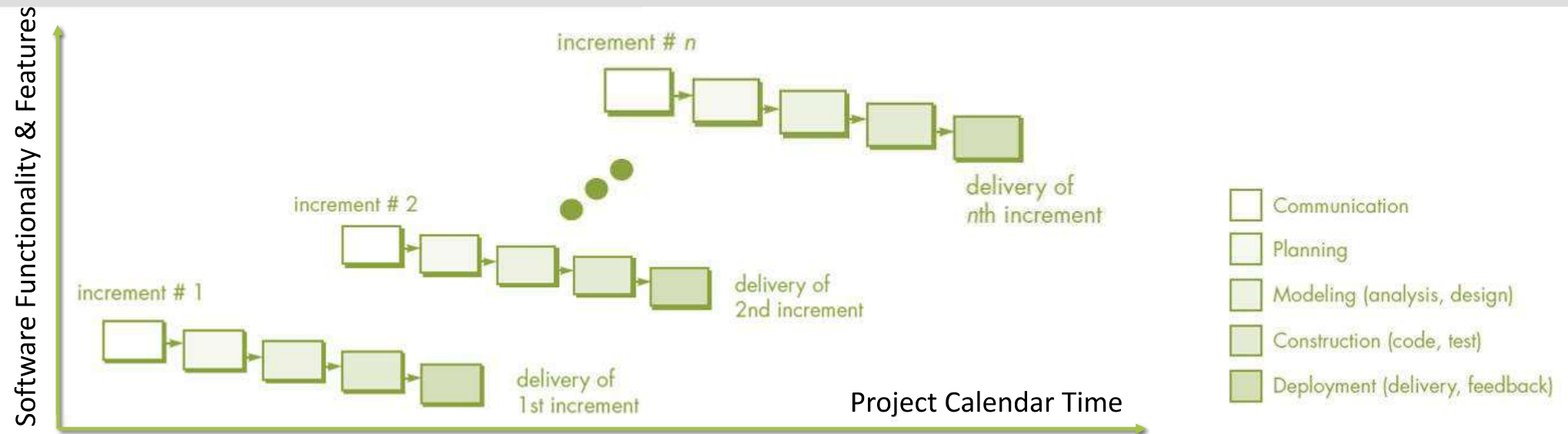
Process Models

- Waterfall Model (Linear Sequential Model)
- Incremental Process Model
- Prototyping Model
- The Spiral Model
- Rapid Application Development Model
- Agile Model

Incremental Process Model

- ▶ There are many situations in which **initial software requirements** are reasonably **well defined**, but the **overall scope of the development** effort prevent a purely linear process.
- ▶ In addition, there may be a **compelling need** to provide a **limited set of software functionality** to users **quickly** and then **refine and expand on that functionality** in later software releases.
- ▶ In such cases, there is a need of a process model that is designed to produce the software in increments.

Incremental Process Model



- ▶ The incremental model **combines** elements of **linear** and **parallel** process flows.
- ▶ This model applies linear sequence in a iterative manner.
- ▶ Initially **core working product** is **delivered**.
- ▶ **Each** linear **sequence** produces deliverable “**increments**” of the software.

Incremental Process Model

e.g. **word-processing software** developed **using the incremental model**

- ▶ It might deliver basic file management, editing and document production functions in the first increment
- ▶ more sophisticated editing in the second increment;
- ▶ spelling and grammar checking in the third increment; and
- ▶ advanced page layout capability in the fourth increment.

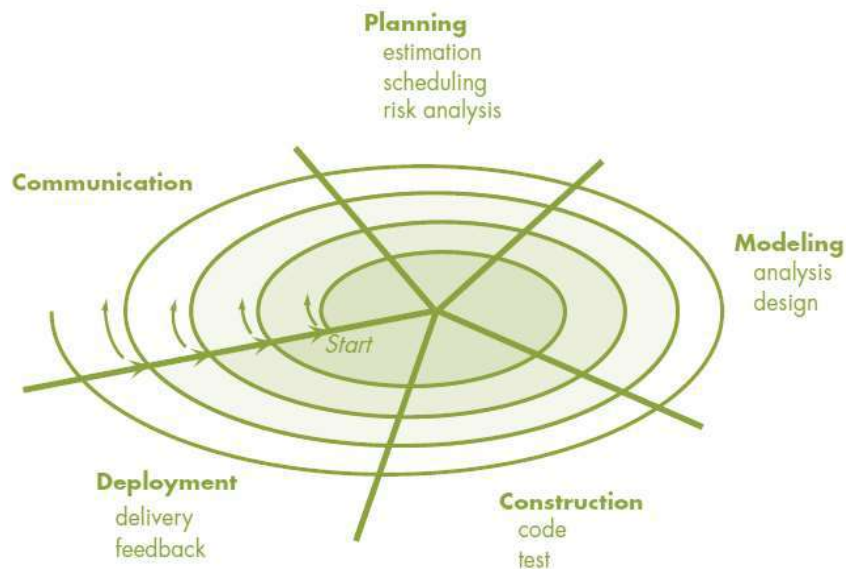
When to use ?

- ▶ When the **requirements** of the **complete** system are clearly **defined** and understood but **staffing is unavailable** for a **complete implementation** by the business deadline.

Advantages

- ▶ Generates **working software quickly** and early during the software life cycle.
- ▶ It is **easier to test** and debug during a smaller iteration.
- ▶ **Customer** can **respond** to each built.
- ▶ **Lowers initial** delivery **cost**.
- ▶ **Easier** to **manage risk** because risky pieces are identified and handled during iteration.

The Spiral Model



- ▶ It provides the **potential** for **rapid development**.
- ▶ Software is developed in a series of evolutionary releases.
- ▶ **Early iteration** release might be **prototype** but **later iterations** provides more **complete version of software**.
- ▶ It is divided into framework activities (C,P,M,C,D). Each activity represent one segment of the spiral
- ▶ **Each pass** through the **planning** region results in **adjustments** to
 - the **project plan**
 - **Cost & schedule** based on feedback

The Spiral Model Cont.

When to use Spiral Model?

- ▶ For development of **large scale / high-risk projects**.
- ▶ When costs and **risk evaluation is important**.
- ▶ Users are **unsure** of their **needs**.
- ▶ **Requirements** are **complex**.
- ▶ New product line.
- ▶ Significant **(considerable) changes** are expected.

Advantages

- ▶ High amount of risk analysis hence, **avoidance of Risk** is enhanced.
- ▶ **Strong approval** and **documentation** control.
- ▶ **Additional functionality** can be **added** at a later date.
- ▶ **Software** is **produced early** in the Software Life Cycle.

Disadvantage

- ▶ Can be **a costly model** to use.
- ▶ Risk analysis **requires highly specific expertise**.
- ▶ Project's success is highly dependent on the risk analysis phase.
- ▶ Doesn't work well for smaller projects.

CO2

1) W5HH

2) Function Point Numerical

3) COCOMO Model Differences and Numerical

4) Risk Categorization

5) RMMM

W⁵HH of Project Management

Boehm suggests **an approach (W⁵HH)** that addresses **project objectives, milestones and schedules, responsibilities, management and technical approaches, and required resources**

Why is the system being developed?

Enables all parties to assess the validity of business reasons for the software work. In another words - does the business purpose justify the expenditure of people, time, and money?

What will be done?

The **answers to these questions** help the team to **establish a project schedule** by **identifying key project tasks** and the **milestones** that are required by the customer

When will it be accomplished?

Project schedule to achieve milestone



W⁵HH of Project Management Cont.

Who is responsible?

Role and responsibility of each member

Where are they organizationally located?

Customer, end user and other stakeholders also have responsibility

How will the job be done technically and managerially?

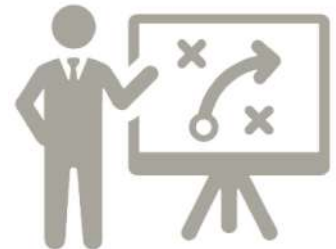
Management and technical strategy must be defined

How much of each resource is needed?

Develop estimation

It is applicable **regardless** of **size** or **complexity** of software **project**

W⁵HH



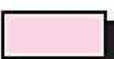
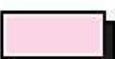
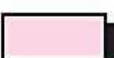
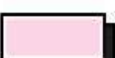
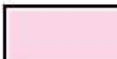
Compute Function Points

FP = Count Total * [0.65 + 0.01 * $\sum(F_i)$]

Count Total is the sum of all FP entries

Fi (i=1 to 14) are complexity value adjustment factors (**VAF**).

Value adjustment factors are used to provide an indication of problem complexity

Information Domain Value	Count		Weighting factor				
			Simple	Average	Complex		
External Inputs (EIs)		×	3	4	6	=	
External Outputs (EOs)		×	4	5	7	=	
External Inquiries (EQs)		×	3	4	6	=	
Internal Logical Files (ILFs)		×	7	10	15	=	
External Interface Files (EIFs)		×	5	7	10	=	
Count total							

Compute Function Points Cont.

Value Adjustment Factors

- F1. Data Communication
- F2. Distributed Data Processing
- F3. Performance
- F4. Heavily Used Configuration
- F5. Transaction Role
- F6. Online Data Entry
- F7. End-User Efficiency
- F8. Online Update
- F9. Complex Processing
- F10. Reusability
- F11. Installation Ease
- F12. Operational Ease
- F13. Multiple Sites
- F14. Facilitate Change

Compute Function Points Cont.

Function Point Calculation Example

Information Domain Value	Count		Weighting factor				
			Simple	Average	Complex		
External Inputs (EIs)	3	×	3	4	6	=	9
External Outputs (EOs)	2	×	4	5	7	=	8
External Inquiries (EQs)	2	×	3	4	6	=	6
Internal Logical Files (ILFs)	1	×	7	10	15	=	7
External Interface Files (EIFs)	4	×	5	7	10	=	20
Count total							50

Compute Function Points Cont.

Used Adjustment Factors and assumed values are,

F09. Complex internal processing = **3**

F10. Code to be reusable = **2**

F13. Multiple sites = **3**

F03. High performance = **4**

F02. Distributed processing = **5**

Project Adjustment Factor (VAF) = 17

$$\text{FP} = \text{Count Total} * [0.65 + 0.01 * \sum(F_i)]$$

$$\text{FP} = [50] * [0.65 + 0.01 * 17]$$

$$\text{FP} = [50] * [0.65 + 0.17]$$

$$\text{FP} = [50] * [0.82] = 41$$

Compute Function Points Cont.

Function Point Calculation Example 2

Study of requirement specification for a project has produced following results

Need for **7 inputs**, **10 outputs**, **6 inquiries**, **17 files** and **4 external interfaces**

Input and **external interface function point** attributes are of **average complexity** and all **other function points** attributes are of **low complexity**

Determine **adjusted function points** assuming complexity **adjustment value is 32**.

Compute Function Points Cont.

Information Domain Value	Count		Weighting factor				
			Simple	Average	Complex		
External Inputs (EIs)	7	×	3	4	6	=	28
External Outputs (EOs)	10	×	4	5	7	=	40
External Inquiries (EQs)	6	×	3	4	6	=	18
Internal Logical Files (ILFs)	17	×	7	10	15	=	119
External Interface Files (EIFs)	4	×	5	7	10	=	28
Count total							233

Value adjustment factors **(VAF) = 32** given

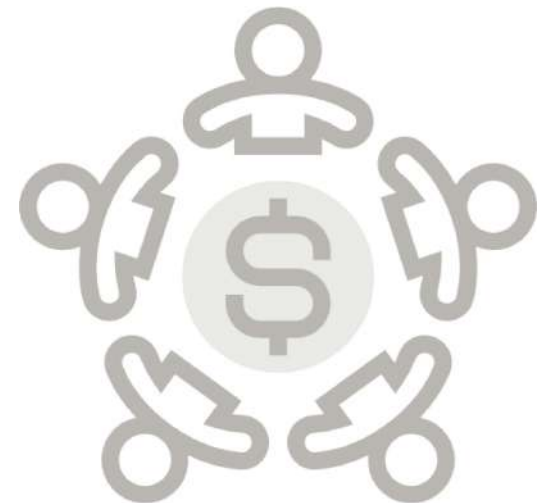
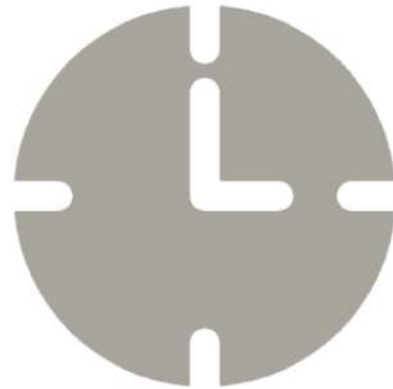
$$\mathbf{FP} = \text{Count Total} * [0.65 + 0.01 * \Sigma(F_i)]$$

$$= \mathbf{233} * [0.65 + 0.01 * \mathbf{32}]$$

$$= \mathbf{233} * 0.97 = \mathbf{226.01}$$

COCOMO Model

- **COCOMO (Constructive Cost Estimation Model)** was proposed by Boehm
- According to Boehm, **software cost estimation** should be done through three stages:
 - Basic COCOMO,
 - Intermediate COCOMO, and
 - Complete COCOMO



Basic COCOMO Model

The **basic COCOMO** model gives an **approximate estimate** of the project parameters

The **basic COCOMO estimation** model is given by the **following expressions**

$$\textit{Effort} = a_1 \times (KLOC)^{a_2} \textit{ PM} \quad \textit{Tdev} = b_1 \times (\textit{Effort})^{b_2} \textit{ Months}$$

- **KLOC** is the estimated size of the software product expressed in Kilo Lines of Code,
- **a_1 , a_2 , b_1 , b_2** are constants for each category of software products,
- **Tdev** is the estimated **time to develop** the software, **expressed in months**,
- **Effort** is the **total effort required to develop** the software **product**, expressed in **person months (PMs)**.

Project	a_1	a_2	b_1	b_2
Organic	2.4	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Basic COCOMO Model Cont.

- The effort estimation is expressed in **units of person-months (PM)**
- It is the **area under the person-month plot** (as shown in fig.)
- An **effort of 100 PM**
 - **does not** imply that **100 persons** should **work for 1 month**
 - **does not** imply that **1 person** should be **employed for 100 months**
 - **it denotes** the **area under** the **person-month curve** (fig.)

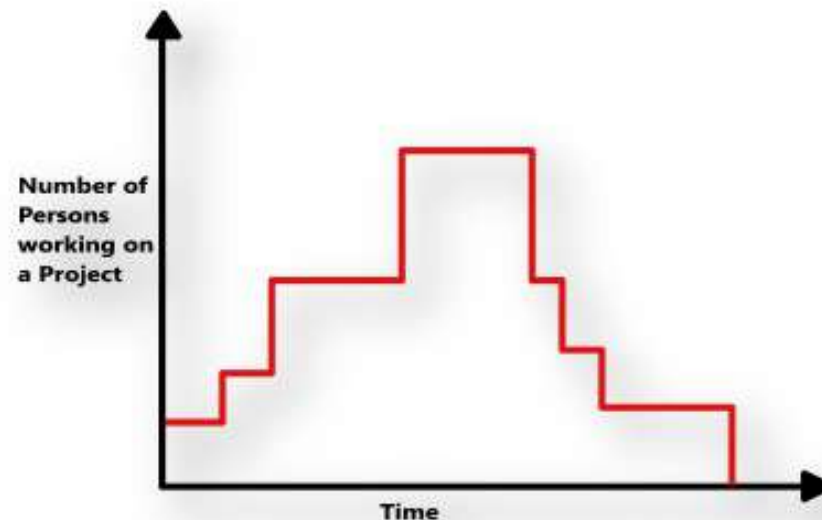
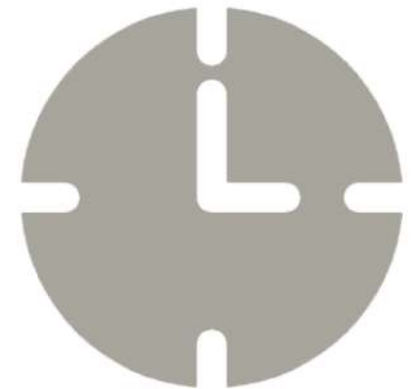


Fig. Person Month Curve

Basic COCOMO Model Cont.

- **Every line of source** text should be **calculated** as **one LOC** **irrespective of** the **actual number of instructions** on that line
- If a **single instruction spans several lines** (say **n lines**), it is **considered** to be **nLOC**
- The values of **a_1 , a_2 , b_1 , b_2** for different categories of products (i.e. organic, semidetached, and embedded) as given by Boehm
- He derived the expressions by examining historical data collected from a large number of actual projects



Basic COCOMO Model Cont.

- **Insight** into the **basic COCOMO** model can be obtained by **plotting the estimated characteristics** for different software sizes
- **Fig.** shows a **plot of estimated effort** versus **product size**
- From fig. we can observe that the **effort** is somewhat **superlinear** in the **size of the software** product
- The **effort** required to develop a product **increases** very **rapidly** with project size

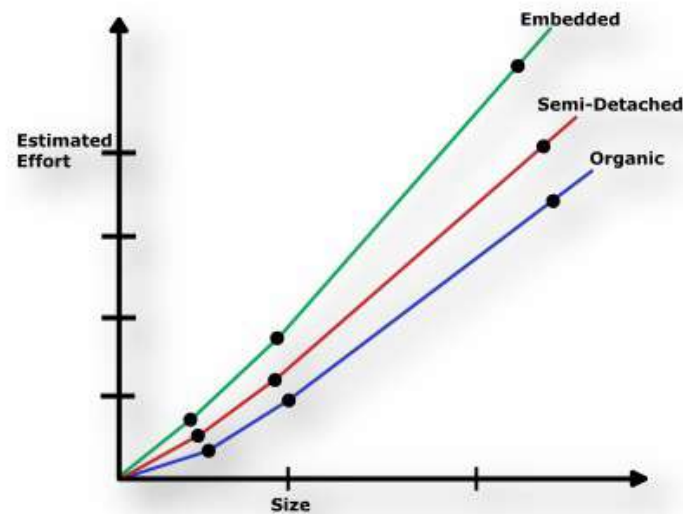
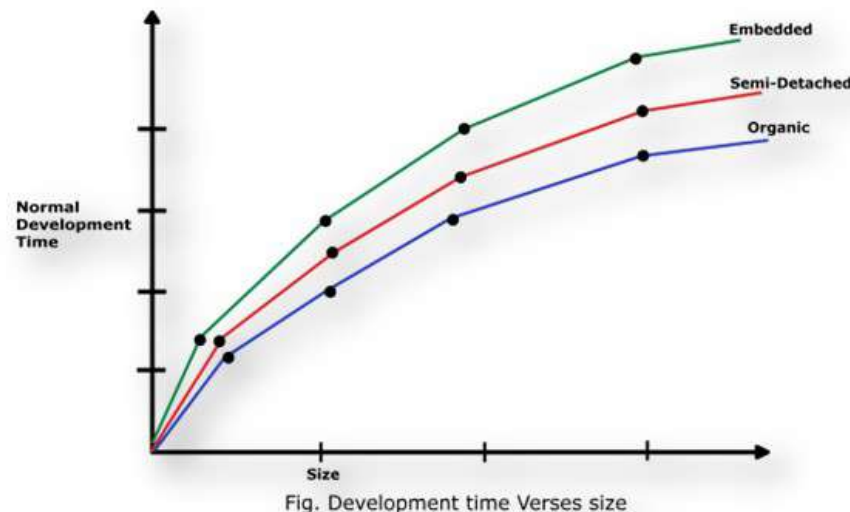


Fig. Effort Verses Product size

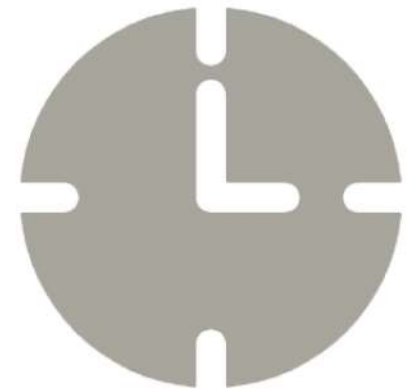
Basic COCOMO Model Cont.

- The **development time versus the product size** in KLOC is plotted in fig.
- From fig., it can be observed that the **development time** is a **sublinear** function of **the size** of the product
- i.e. when the **size of the product** increases by **two times**, the time to develop the **product does not double** but **risers moderately**
- From fig., it can be observed that the development time is roughly the same for all the three categories of products



Basic COCOMO Model Cont.

- **Effort** and the **duration estimations** obtained **using** the **COCOMO** model are called as **nominal effort** estimate and nominal **duration estimate**
- The term **nominal implies** that
 - if anyone **tries to complete the project** in a **time shorter** than the **estimated** duration, then the **cost will increase drastically**
 - But, if anyone **completes the project over a longer period** of time than the **estimated**, then there is almost **no decrease** in the **estimated cost value**



Basic COCOMO Model Cont.

Example:

Assume that the **size** of an **organic type** software product **has been estimated** to be **32,000 lines of source code**. Assume that the **average salary** of software **engineers** be **Rs. 15,000/- per month**. **Determine** the **effort required** to develop the software product **and** the **nominal development time**

$$\textit{Effort} = a_1 \times (KLOC)^{a_2} \textit{ PM}$$

$$= 2.4 \times (32)^{1.05} \textit{ PM}$$

$$= 91 \textit{ PM}$$

$$\textit{Tdev} = b_1 \times (\textit{Effort})^{b_2} \textit{ Months}$$

$$= 2.5 \times (91)^{0.38} \textit{ Months}$$

$$= 14 \textit{ Months}$$

Cost required to develop the product = 14 x 15000 = Rs. **2,10,000/-**

Intermediate COCOMO model

- The **basic COCOMO** model **assumes** that **effort** and **development time** **are functions** of the **product size alone**
- However, a **host of other** project **parameters** besides the product size **affect** the **effort required** to develop the product as well as the **development time**
- Therefore, **in order to obtain an accurate estimation** of the effort and project **duration**, the **effect** of all relevant **parameters** must be **taken into account**
- The **intermediate COCOMO** model **recognizes this fact** and refines the initial estimate obtained using the basic COCOMO expressions **by using a set of 15 cost drivers (multipliers)** based on **various attributes** of software development
 - For example, if modern programming practices are used, the initial estimates are scaled downward by multiplication with a cost driver having a value less than 1

Intermediate COCOMO model Cont.

- It is **requires** the project manager **to rate** these **15** different **parameters** for a particular **project** on a **scale of one to three**.
- Then, **depending** on these **ratings**, **appropriate cost** driver **values** which should be **multiplied with the initial estimate** obtained using the basic COCOMO.

The cost drivers can be **classified** as being attributes
of the following items

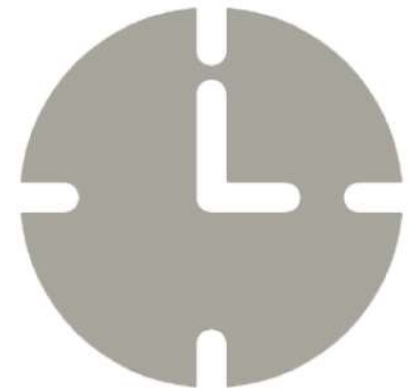
Product: The characteristics of the product that are considered include the **inherent complexity** of the product, **reliability requirements** of the product, etc.

Computer: Characteristics of the computer that are considered include the **execution speed** required, **storage space** required etc.

Intermediate COCOMO model Cont.

Personnel: The attributes of development personnel that are considered include the **experience level of personnel**, **programming capability**, **analysis capability**, etc.

Development Environment: Development environment attributes capture the **development facilities available** to the developers. An important parameter that is considered is the **sophistication of the automation (CASE) tools used** for software development



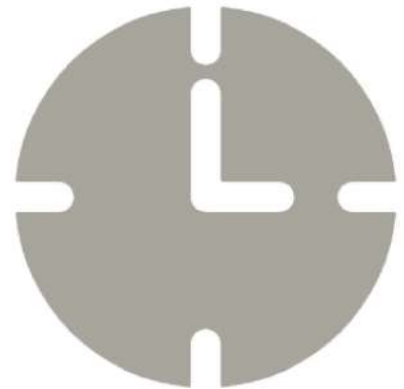
Complete COCOMO model

- A major **shortcoming** of both the **basic** and **intermediate COCOMO** models is that they **consider** a software product **as a single homogeneous entity**
- Most **large systems** are **made** up several **smaller sub-systems**
- These **sub-systems** may have widely **different characteristics**
 - E.g., **some sub-systems** may be considered as **organic type**, some **semidetached**, and some **embedded**
 - Also for **some subsystems** the **reliability requirements** may be **high**, for some the **development team** might have no previous **experience** of similar development etc.
- The **complete COCOMO** model **considers** these differences in **characteristics** of the **subsystems** and **estimates** the effort and development time **as the sum of the estimates for the individual subsystems**



Complete COCOMO model Cont.

- The **cost** of each **subsystem** is **estimated separately**
- This approach **reduces** the **margin of error** in the final **estimate**



Risk

A risk is a **potential (probable) problem** – which might **happen and might not**



Conceptual definition of risk

- Risk **concerns future happenings**
- Risk **involves change** in mind, **opinion**, **actions**, **places**, etc.
- Risk **involves** choice and the **uncertainty** that choice entails

Two characteristics of risk

Uncertainty

The risk **may** or **may not happen**, so there are no 100% risks (some of those may called constraints)

Loss

If the risk becomes a **reality** and **unwanted consequences** or **losses** occur

Risk Categorization: Approach-1

■ Project risks

- They **threaten** the **project plan**
- If they become real, it is likely that the **project schedule will slip** and that **costs will increase**

■ Technical risks

- They **threaten the quality** and **timeliness** of the software to be produced
- If they become real, **implementation may become difficult** or impossible

■ Business risks

- They **threaten** the **feasibility** of the **software** to be built
- If they become real, they **threaten the project** or the **product**



Risk Categorization: Approach-1

Sub-categories of Business risks

- **Market risk**
 - Building an **excellent product** or system that **no one really wants**
- **Strategic risk**
 - Building a **product that no longer fits into** the overall **business strategy** for the company
- **Sales risk**
 - Building a **product** that the **sales force doesn't understand** how to **sell**
- **Management risk**
 - **Losing** the **support** of **senior management** due to a change in focus or a change in people
- **Budget risk**
 - **Losing budgetary** or **personnel commitment**



Risk Categorization: Approach-2

■ Known risks

- Those **risks** that can be **uncovered after careful evaluation** of
 - the **project plan**,
 - the **business and technical environment** in which the project is being developed, and
 - other **reliable information sources** (Ex. unrealistic delivery date)

■ Predictable risks

- Those **risks** that are **deduced** (draw conclusion) from **past project** experience (Ex. past turnover)

■ Unpredictable risks

- Those **risks** that can and do **occur**, but are **extremely difficult** to **identify** in **advance**



Risk Strategies (Reactive vs. Proactive)

■ Reactive risk strategies

- "Don't worry, I will think of something".
- The majority of software teams and managers rely on this approach
- Nothing is done about risks until something goes wrong
- The team then flies into action in an attempt to correct the problem rapidly (fire fighting)
- Crisis management is the choice of management techniques

■ Proactive risk strategies

- Steps for risk management are followed
- Primary objective is to avoid risk and to have an emergency plan in place to handle unavoidable risks in a controlled and effective manner



RMMM

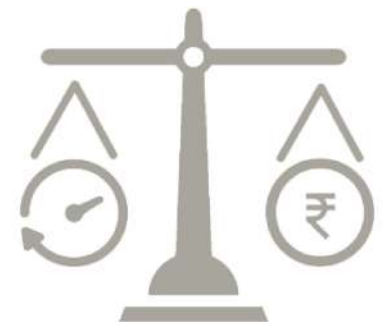
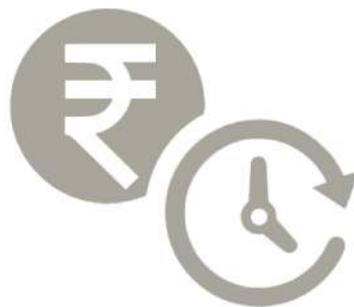
- **RMMM** - Mitigation, Monitoring, and Management
- An **effective strategy** for **dealing with risk** must consider three issues
 - Risk **mitigation** (i.e., **avoidance**)
 - Risk **monitoring**
 - Risk **management** and **contingency planning**

RMMM

Risk Mitigation is a **problem avoidance activity**

Risk Monitoring is a **project tracking activity**

Risk Management includes **contingency plans** that risk will occur



Risk Mitigation

- Risk **mitigation (avoidance)** is the **primary strategy** and is achieved through a plan

For Ex., Risk of high staff turnover

To mitigate this risk, you would develop a strategy for reducing turnover.

The possible steps to be taken are:

- **Meet with current staff** to determine **causes for turnover** (e.g., poor working conditions, low pay, and competitive job market)
- **Mitigate** those **causes** that are **under your control** before the project starts
- Once the **project commences**, assume **turnover will occur** and develop **techniques to ensure continuity** when **people leave**
- **Organize project teams** so that **information** about **each development activity** is **widely dispersed**



Risk Mitigation Cont.

- **Define** work **product standards** and establish mechanisms to **be sure** that all **models** and **documents** are **developed** in a **timely manner**
- Conduct **peer reviews of all work** (so that more than one person is “up to speed”).
- Assign **a backup staff member** for every **critical technologist**



RMMM PLAN

- The RMMM **PLAN documents** all work performed as part of **risk analysis** and used by the project manager as part of the **overall project plan**
- Some software teams do not develop a formal RMMM document, rather each risk is documented individually using a **Risk information sheet (RIS)**
- In most cases, RIS is maintained using a database system.
- So Creation and information entry, priority ordering, searches and other analysis may be accomplished easily.
- The format of RIS is describe in diagram

Risk information sheet (RIS)

Risk information sheet			
Risk ID: P02-4-32	Date: 5/9/02	Prob: 80%	Impact: high
Description: Only 70 percent of the software components scheduled for reuse will, in fact, be integrated into the application. The remaining functionality will have to be custom developed.			
Refinement/context: Subcondition 1: Certain reusable components were developed by a third party with no knowledge of internal design standards. Subcondition 2: The design standard for component interfaces has not been solidified and may not conform to certain existing reusable components. Subcondition 3: Certain reusable components have been implemented in a language that is not supported on the target environment.			
Mitigation/monitoring: 1. Contact third party to determine conformance with design standards. 2. Press for interface standards completion; consider component structure when deciding on interface protocol. 3. Check to determine number of components in subcondition 3 category; check to determine if language support can be acquired.			
Management/contingency plan/trigger: RE computed to be \$20,200. Allocate this amount within project contingency cost. Develop revised schedule assuming that 18 additional components will have to be custom built; allocate staff accordingly. Trigger: Mitigation steps unproductive as of 7/1/02			
Current status: 5/12/02: Mitigation steps initiated.			
Originator: D. Gagne		Assigned: B. Laster	

CO3:

- 1) SRS (including Characteristics)
- 2) Class Diagram of Bank & Library Management System
- 3) Activity & Swimlane Diagram for Book Issue
- 4) Cohesion & Coupling

SRS (Software Requirements Specification)

Software Requirement Specification (SRS) is a **document that completely describes what the proposed software should do** without describing how software will do it.

SRS is also helping the clients to understand their own needs.

SRS Contains

A **complete information** description

A **detailed functional** description

A representation of **system behaviour**

An indication of **performance requirements and design** constraints

Appropriate **validation criteria**

Other information suitable to requirements

Characteristics of a Good SRS

- ▶ SRS should be **accurate, complete, efficient, and of high quality**, so that it does not affect the entire project plan.
- ▶ An SRS is **said to be of high quality when** the developer and user **easily understand** the prepared document.
- ▶ **Characteristics of a Good SRS:**

Correct

SRS is correct when **all user requirements are stated** in the requirements document.

Note that there is **no specified tool or procedure to assure the correctness** of SRS.

Unambiguous

SRS is unambiguous when **every stated requirement has only one interpretation**.

Complete

SRS is complete when the **requirements clearly define what the software is required to do**.

Characteristics of a Good SRS Cont.

Ranked for Importance/Stability

All requirements are not equally important, hence **each requirement is identified to make differences** among other requirements.

Stability implies the probability of changes in the requirement in future.

Modifiable

The requirements of the user can change, hence requirements document should be created in such a manner that those **changes can be modified easily**.

Traceable

SRS is traceable when the **source of each requirement is clear** and facilitates the reference of each requirement in future.

Verifiable

SRS is verifiable when the **specified requirements can be verified with a cost-effective process** to check whether the final software meets those requirements.

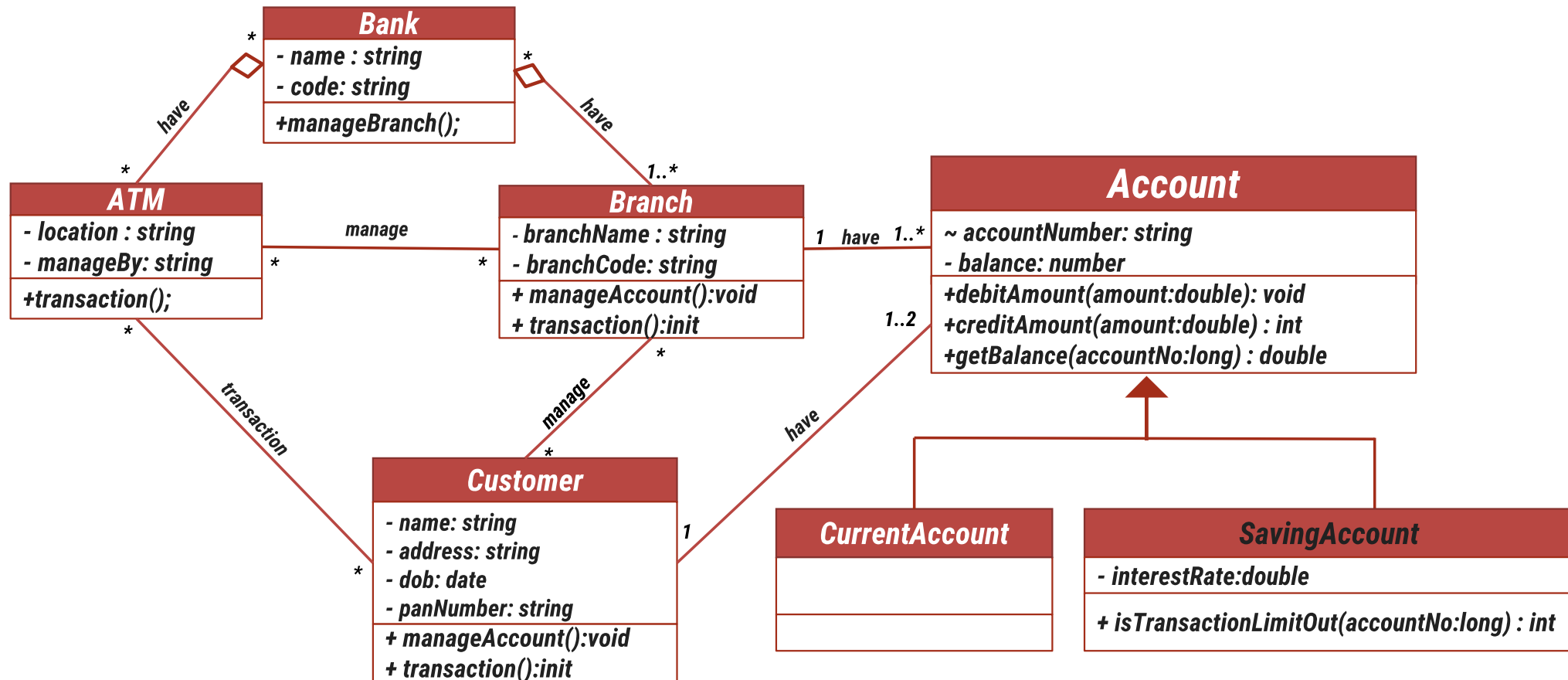
Consistent

SRS is consistent when the **subsets of individual requirements defined do not conflict** with each other.

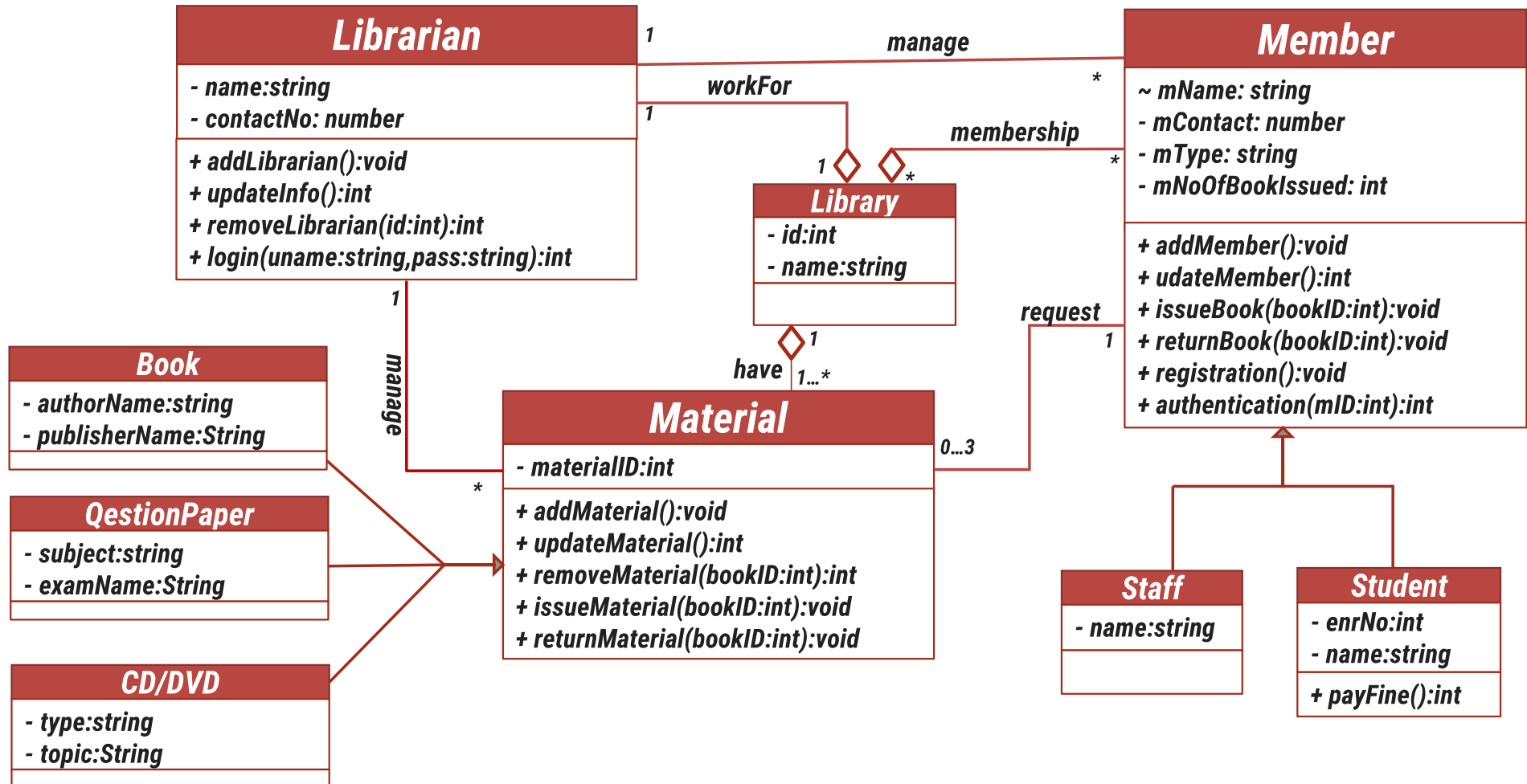
Problems Without SRS

- ▶ Without developing the SRS document, the **system would not be properly implemented according to customer needs.**
- ▶ Software developers would not know whether what they are developing is **what exactly is required by the customer.**
- ▶ Without SRS, it will be very difficult for the **maintenance engineers to understand the functionality** of the system.
- ▶ It will be very difficult for **user document writers to write the users' manuals properly** without understanding the SRS.

Class Diagram Of Bank Management System



Class Diagram Of Library Management System



Guideline for Activity Diagram

Level diagrams

- ▶ *Activities on a diagram should be at a consistent level of details.*
- ▶ *Place additional details for an activity in a separate diagram.*

Be careful with branches and conditions

- ▶ *If there are conditions, at **last one must be satisfied** when an activity completes, consider using an [else] condition.*
- ▶ *It is **possible for multiple conditions to be satisfied otherwise this is an error condition.***

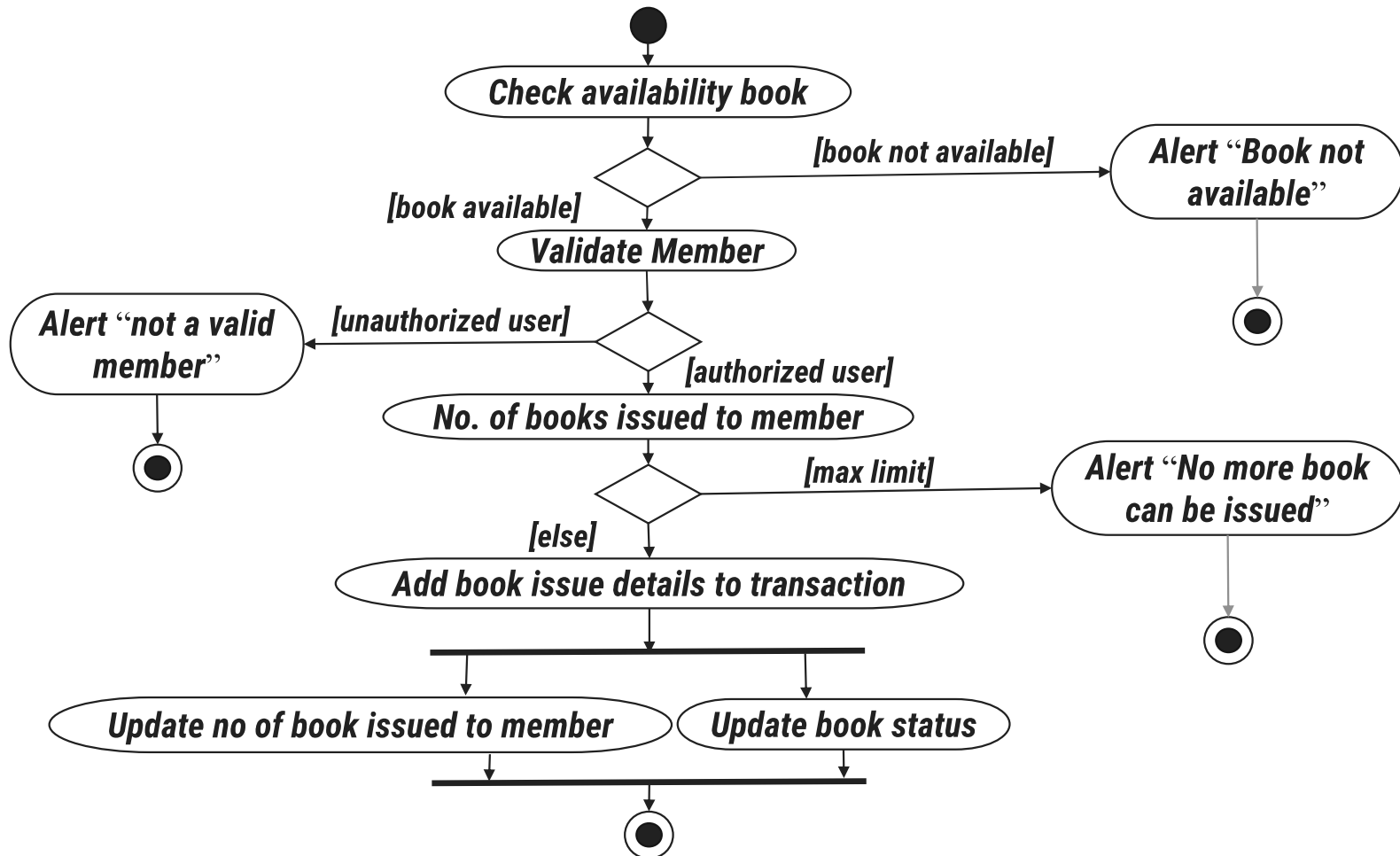
Be careful with concurrent activities

- ▶ *Means that the **activities can complete in any order** and still yield an acceptable result.*
- ▶ ***Before a merge can happen, all inputs must first complete***

How to Draw an Activity Diagram

- ▶ **Step 1: Identify the various activities** and actions your business process or system
- ▶ **Step 2: Find a flow among the activities**
- ▶ For e.g. in library management system, **book issue** is a one business process or a **function**. Show we prepare a activity diagram for Book issue.
- ▶ **Various activity in book issue process like...**
 - Check availability of book
 - Validate the member
 - Check No. of books issued by member
 - Add book issue details to transaction
 - Update no of book issued by member
 - Update book status.

Activity Diagram for Book Issue



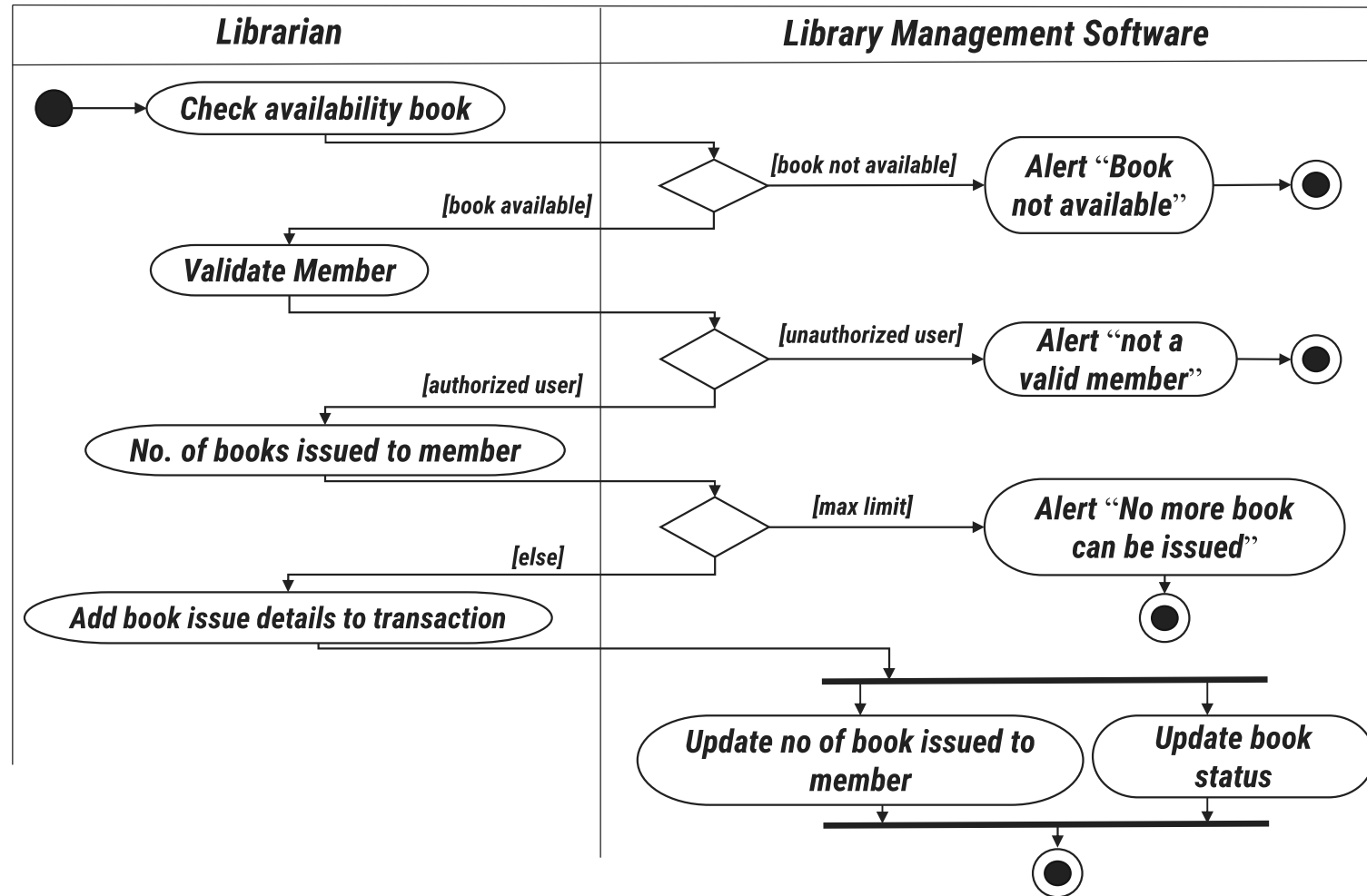
Swimlane Diagram

- ▶ In a business model, it is often useful to know **which human department is responsible** for an activity.
- ▶ When design of the system is complete, the activity will be **assigned to a person/department**, but at a high level it is **sufficient to partition the activities** among departments.
- ▶ You can show such a partitioning with an activity diagram by **dividing in to columns and lines**.
- ▶ **Each column is called swim-lane** by analogy to a swimming pool.
- ▶ Placing an **activity** within a **particular** swim-lane **indicates** that is **performed by a person/department**.
- ▶ Lines across swim-lane **boundaries indicate interaction** among different person/department.

How to Draw a Swimlane Diagram

- ▶ ***Step 1: Identify the various activities and actions your business process or system***
- ▶ ***Step 2: Figure out which person/departments are responsible for the completion of activity.***
- ▶ ***Step 3: Figure out in which order the actions are processed.***
- ▶ ***Step 4: Figure out who is responsible for each action and assign them a swimlane and group each action they are responsible for under them***

Swimlane Diagram for Book Issue

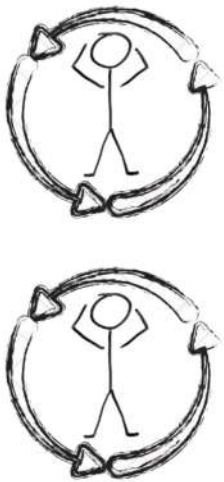


Cohesion & Coupling

A **good software design** implies **clean decomposition** of the **problem into modules**, and the **neat arrangement** of these **modules** in a **hierarchy**.

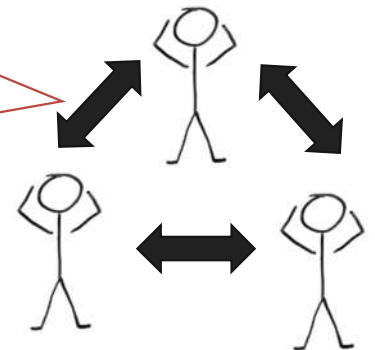


The primary **characteristics** of **neat module decomposition** are **high cohesion** and **low coupling**.



A **cohesive module** performs a single task, requiring little interaction with other components.

A **Coupling** is an indication of the relative interdependence among modules.



Cohesion

- ▶ Cohesion is an **indication** of the **relative functional strength** of a module.
- ▶ A **cohesive module** performs a **single task, requiring little interaction** with other components.
- ▶ Stated simply, a **cohesive module** should (ideally) **do just one thing**.
- ▶ A module having **high cohesion** and **low coupling** is said to be **functionally independent** of other modules.
- ▶ By the term functional independence, we mean that a **cohesive module performs a single task or function**.

Classification of Cohesion

Classification of Cohesion	Coincidental	 Low
	Logical	
	Temporal	
	Procedural	
	Communicational	
	Sequential	
	Functional	High

Classification of Cohesion cont.

Coincidental cohesion

- ▶ A module is said to have coincidental cohesion, if it performs a set of tasks that relate to each other very loosely.
- ▶ In this case, the module contains a random collection of functions.
- ▶ It is likely that the functions have been put in the module out of pure coincidence without any thought or design.
- ▶ For Ex., in a transaction processing system (TPS), the get-input, print-error, and summarize-members functions are grouped into one module
- ▶ At the outer layer, components service user interface operations.

Logical cohesion

- ▶ A module is said to be logically cohesive, if all elements of the module perform similar operations.
- ▶ For Ex., error handling, data input, data output, etc.
- ▶ An example of logical cohesion is the case where a set of print functions generating different output reports are arranged into a single module. In this case, the module contains a random collection of functions.

Classification of Cohesion cont.

Temporal cohesion

- ▶ When a module **contains functions** that are **related by the fact** that all the **functions** must be **executed** in the **same time span**.
- ▶ For Ex., the set of functions responsible for initialization, start-up, shutdown of some process, etc.

Communicational cohesion

- ▶ If **all functions** of the module **refer** to the **same data structure**
- ▶ For Ex., the set of functions defined on an array or a stack.

Procedural cohesion

- ▶ If the set of **functions** of the module are **all part** of a **procedure** (algorithm) in which **certain sequence of steps** **have to be carried out** for **achieving an objective**
- ▶ For Ex., the algorithm for decoding a message.

Classification of Cohesion cont.

Sequential cohesion

- ▶ If the **elements** of a **module form** the parts of **sequence**, where the **output from one element of the sequence is input to the next**.
- ▶ For Ex., In a Transaction Processing System, the get-input, validate-input, sort-input functions are grouped into one module.

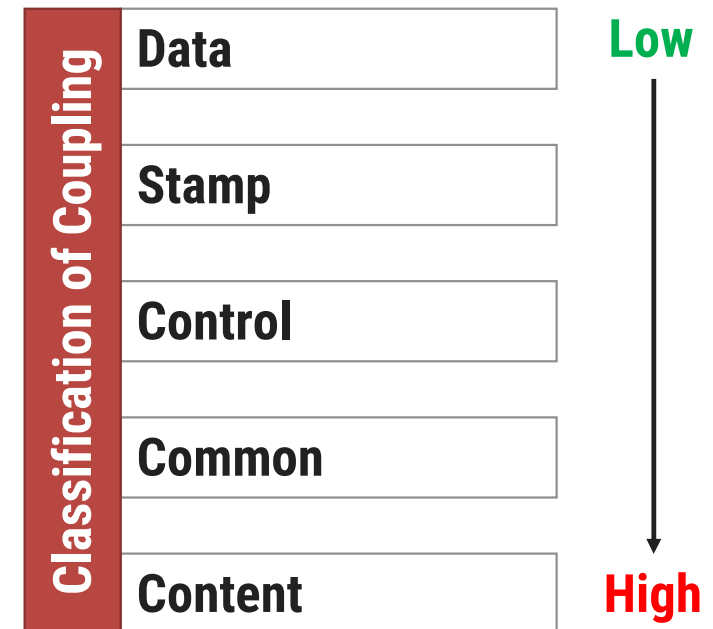
Functional cohesion

- ▶ If different **elements** of a module **cooperate to achieve a single function**.
- ▶ For Ex., A module containing all the functions required to manage employees' pay-roll exhibits functional cohesion.

Coupling

- ▶ **Coupling** between two modules is a **measure of the degree of interdependence** or interaction **between the two modules**.
- ▶ A module having high cohesion and low coupling is said to be functionally independent of other modules.
- ▶ If **two modules interchange large amounts of data**, then they are **highly interdependent**.
- ▶ The degree of coupling between two modules depends on their interface complexity.
- ▶ The interface complexity is basically determined by the number of types of parameters that are interchanged while invoking the functions of the module.

Classification of Coupling



Classification of Coupling Cont.

Data coupling

- ▶ Two modules are data coupled, if **they communicate through a parameter**.
- ▶ An example is an elementary (primal) data item passed as a parameter between two modules, e.g. an integer, a float, a character, etc.

Control coupling

- ▶ If **data** from **one** module **is used to direct the order of instructions** execution **in another**.
- ▶ An example of control coupling is a flag set in one module and tested in another module

Stamp coupling

- ▶ This is a special case (or extension) of data coupling
- ▶ Two modules ("A" and "B") exhibit stamp coupling if **one passes** directly to the other a **composite data item** - such as a record (or structure), array, or (pointer to) a list or tree.
- ▶ This occurs when **ClassB** is **declared as a type** for an argument of an **operation of ClassA**

Classification of Coupling Cont.

Common coupling

- ▶ If **data** from **one** module **is used to direct the order of instructions** execution **in another**.
- ▶ An example of control coupling is a flag set in one module and tested in another module.

Content coupling

- ▶ Content coupling occurs when **one component secretly modifies data** that is **internal to another component**.
- ▶ This violates information hiding – a basic design concept
- ▶ Content coupling exists between two modules, if they share code.

CO4:

- 1) Black & White Box Testing
- 2) Re-Engineering
- 3) Forward & Reverse Engineering
(Difference)
- 4) SCM and it's Layers

Views of Test Objects

Black Box Testing

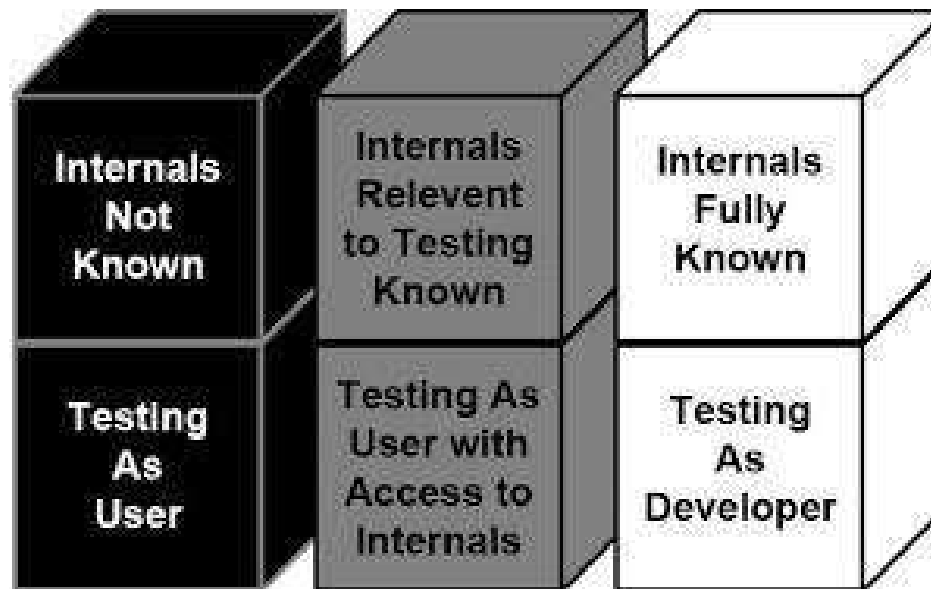
Close Box Testing
Testing based only on
specification

White Box Testing

Open Box Testing
Testing based on actual
source code

Grey Box Testing

Partial knowledge of
source code



Black Box Testing

- ▶ Also known as **specification-based testing**
- ▶ **Tester** has **access** only to **running code** and the **specification** it is supposed to satisfy
- ▶ **Test cases** are **written with no knowledge of internal workings** of the code
- ▶ **No access** to **source code**
- ▶ So **test cases don't worry** about **structure**
- ▶ **Emphasis** is only on **ensuring** that the **contract is met**

Advantages

- ▶ **Scalable;** not dependent on size of code
- ▶ Testing **needs no knowledge** of **implementation**
- ▶ **Tester** and **developer** can be **truly independent** of each other
- ▶ **Tests** are **done** with **requirements in mind**
- ▶ Does **not excuse inconsistencies** in the **specifications**
- ▶ **Test cases** can be **developed** in **parallel** with **code**

Black Box Testing Cont.

Disadvantages

- ▶ **Examine pre-condition**, and **identify** equivalence classes
- ▶ **All possible inputs** such that all classes **are covered**
- ▶ **Apply the specification to input** to write down **expected output**

Test Case Design

- ▶ **Test size** will **have to be small**
- ▶ **Specifications** must be **clear, concise, and correct**
- ▶ May **leave** many **program paths untested**
- ▶ **Weighting** of program **paths** is **not possible**



Black Box Testing Cont.

- ▶ **Exhausting testing** is not always possible when there is a large set of input combinations, because of **budget** and **time** constraint.
- ▶ The **special techniques** are needed which **select test-cases smartly** from the **all combination of test-cases** in such a way that **all scenarios** are covered

Two techniques are used

1 Equivalence Partitioning

2 Boundary Value Analysis (BVA)

Equivalence Partitioning

- ▶ **Input data** for a program unit usually **falls into** a **number of partitions**, e.g. all negative integers, zero, all positive numbers
- ▶ **Each partition** of input data **makes the program behave** in a **similar way**
- ▶ **Two test cases** based on members from the **same partition** is likely to **reveal the same bugs**



Equivalence Partitioning (Black Box Testing)

By **identifying** and **testing one member of each partition** we gain '**good**' coverage with '**small**' number of test cases

Testing one member of a **partition** should **be as good as testing any member** of the partition

Example - Equivalence Partitioning

For binary search the **following partitions** exist

- **Inputs** that *conform* to *pre-conditions*
- **Inputs** where the *precondition* is *false*
- **Inputs** where the *key element* is *a member* of the *array*
- **Inputs** where the *key element* is *not a member* of the *array*

► **Pick specific conditions** of the array

- The *array* has a *single value*
- *Array length* is *even*
- *Array length* is *odd*

Equivalence Partitioning (Black Box Testing) Cont.

Example - Equivalence Partitioning

- ▶ Example: Assume that we have to test field which accepts SPI (Semester Performance Index) as input (SPI range is 0 to 10)

SPI

* Accepts value 0 to 10

Equivalence Partitioning		
Invalid	Valid	Invalid
≤ -1	0 to 10	≥ 11

- ▶ **Valid Class: 0 – 10**, pick any one input test data from 0 to 10
- ▶ **Invalid Class 1: ≤ -1** , pick any one input test data less than or equal to -1
- ▶ **Invalid Class 2: ≥ 11** , pick any one input test data greater than or equal to 11

Boundary Value Analysis (BVA) (Black Box Testing)

- ▶ It arises from the **fact that most program fail at input boundaries**
- ▶ Boundary testing is the **process** of **testing between extreme ends** or boundaries between partitions of the input values.
- ▶ In Boundary Testing, Equivalence Class Partitioning plays a good role
- ▶ **Boundary Testing** comes after the **Equivalence Class Partitioning**
- ▶ The basic idea in boundary value testing is to **select input variable values at their:**

Just below the minimum	Minimum	Just above the minimum
Just below the maximum	Maximum	Just above the maximum



Boundary Value Analysis (BVA) (Black Box Testing)

- ▶ Suppose **system asks** for “a **number between 100 and 999 inclusive**”

- ▶ The **boundaries** are **100** and **999**

- ▶ We therefore **test for values**

99	100	101	999	999	1000
Lower boundary			Upper boundary		

BVA - Advantages

- ▶ The BVA is **easy to use and remember** because of the **uniformity of identified tests** and the automated nature of this technique.
- ▶ One can **easily control the expenses** made on the testing by controlling the number of identified test cases.
- ▶ BVA is the **best approach** in cases where the **functionality** of a software is based on **numerous variables** representing physical quantities.
- ▶ The technique **is best at user input troubles** in the software.
- ▶ The **procedure and guidelines are crystal clear** and easy when it comes to determining the test cases through BVA.
- ▶ The **test cases** generated through BVA are **very small**.

Boundary Value Analysis (BVA) (Black Box Testing) Cont.

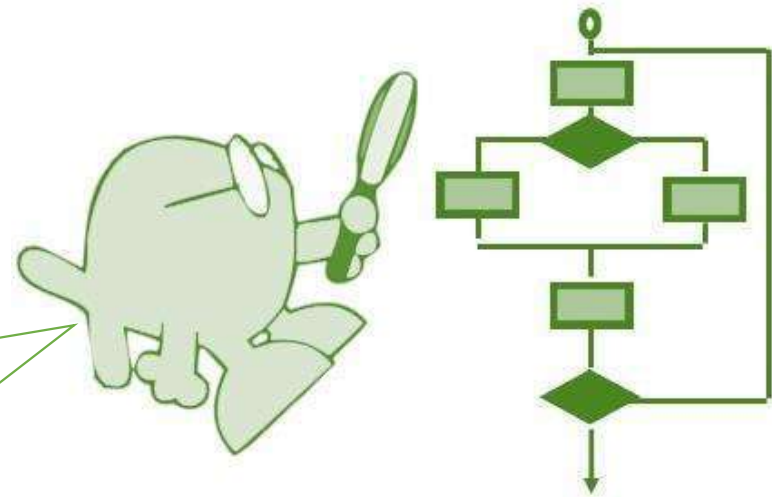
BVA - Disadvantages

- ▶ This technique **sometimes fails** to test **all the potential input** values. And so, the **results are unsure**.
- ▶ The **dependencies** with **BVA** **are not tested between two inputs**.
- ▶ This technique **doesn't fit** well when it comes to **Boolean Variables**.
- ▶ It **only works** well with **independent variables** that depict quantity.

White Box Testing

- ▶ Also known as **structural testing**
- ▶ White Box Testing is a software testing **method** in which the **internal structure/design/implementation** of the module being tested **is known to the tester**
- ▶ Focus is on **ensuring** that even **abnormal invocations** are **handled gracefully**
- ▶ Using white-box testing methods, you can **derive test cases** that
 - **Guarantee** that all **independent paths** within a module have been **exercised at least once**
 - **Exercise** all **logical decisions** on their true and false sides
 - **Execute** all **loops** at their boundaries
 - **Exercise internal data structures** to **ensure** their **validity**

...our goal is to **ensure** that **all statements and conditions** have been **executed at least once** ...



White Box Testing Cont.

► It is applicable to the following levels of software testing

- **Unit Testing:** For testing *paths within a unit*
- **Integration Testing:** For testing *paths between units*
- **System Testing:** For testing *paths between subsystems*

Advantages

- **Testing** can be **commenced** at an **earlier stage** as one need not wait for the GUI to be available.
- **Testing** is **more thorough**, with the possibility of **covering most paths**

Disadvantages

- Since **tests** can be very **complex**, **highly skilled resources** are **required**, with thorough **knowledge** of **programming** and **implementation**
- **Test script maintenance** can be a **burden**, if the **implementation** changes too frequently
- Since this method of testing is closely tied with the application being testing, **tools to cater to every kind of implementation/platform** may not be readily available

White-box testing strategies

- ▶ One white-box testing strategy is said to be stronger than another strategy, if all types of errors detected by the first testing strategy is also detected by the second testing strategy, and the second testing strategy additionally detects some more types of errors.

White-box testing strategies

1 **Statement** coverage

2 **Branch** coverage

3 **Path** coverage

Statement coverage

- ▶ It aims to design test cases so that **every statement in a program is executed at least once**
- ▶ Principal idea is **unless a statement is executed**, it is very **hard to determine if an error exists** in that statement
- ▶ Unless a statement is executed, it is very difficult to observe whether it causes failure due to some illegal memory access, wrong result computation, etc.

White-box testing strategies Cont.

Consider the Euclid's **GCD computation** algorithm

```
int compute_gcd(x, y)
int x, y;
{
1   while (x != y){
2   if (x>y) then
3       x = x - y;
4   else y = y - x;
5   }
6   return x;
}
```

By choosing the test set **{(x=3, y=3), (x=4, y=3), (x=3, y=4)}**, we can exercise the program such that all statements are executed at least once.

White-box testing strategies Cont.

Branch coverage

- ▶ In the branch coverage based testing strategy, **test cases are designed to make each branch condition to assume true and false values in turn**
- ▶ It is also known as **edge Testing** as in this testing scheme, each edge of a program's control flow graph is traversed at least once
- ▶ Branch coverage **guarantees statement coverage**, so it is stronger strategy compared to Statement Coverage.

Path Coverage

- ▶ In this strategy test cases are executed in such a way that **every path is executed at least once**
- ▶ All possible control paths taken, including
 - All loop paths taken **zero, once** and **multiple items** in technique
 - The **test case** are **prepared** based on the **logical complexity measure** of the **procedure design**
- ▶ **Flow graph, Cyclomatic Complexity** and **Graph Metrics** are used to arrive at basis path.

White-box testing strategies Cont.

Branch coverage

- ▶ In the branch coverage based testing strategy, **test cases are designed to make each branch condition to assume true and false values in turn**
- ▶ It is also known as **edge Testing** as in this testing scheme, each edge of a program's control flow graph is traversed at least once
- ▶ Branch coverage **guarantees statement coverage**, so it is stronger strategy compared to Statement Coverage.

Path Coverage

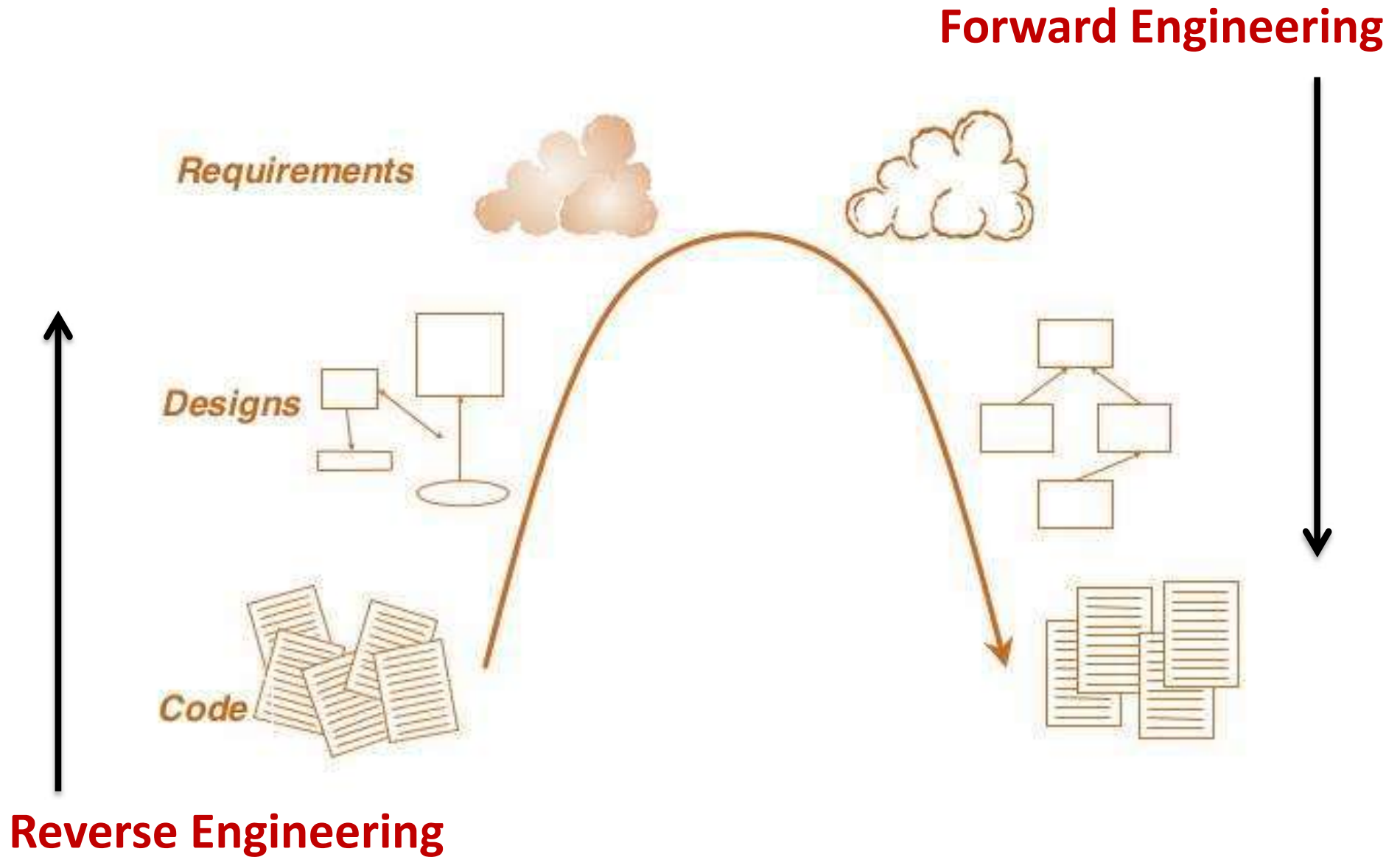
- ▶ In this strategy test cases are executed in such a way that **every path is executed at least once**
- ▶ All possible control paths taken, including
 - All loop paths taken **zero, once** and **multiple items** in technique
 - The **test case** are **prepared** based on the **logical complexity measure** of the **procedure design**
- ▶ **Flow graph, Cyclomatic Complexity** and **Graph Metrics** are used to arrive at basis path.

Re-Engineering

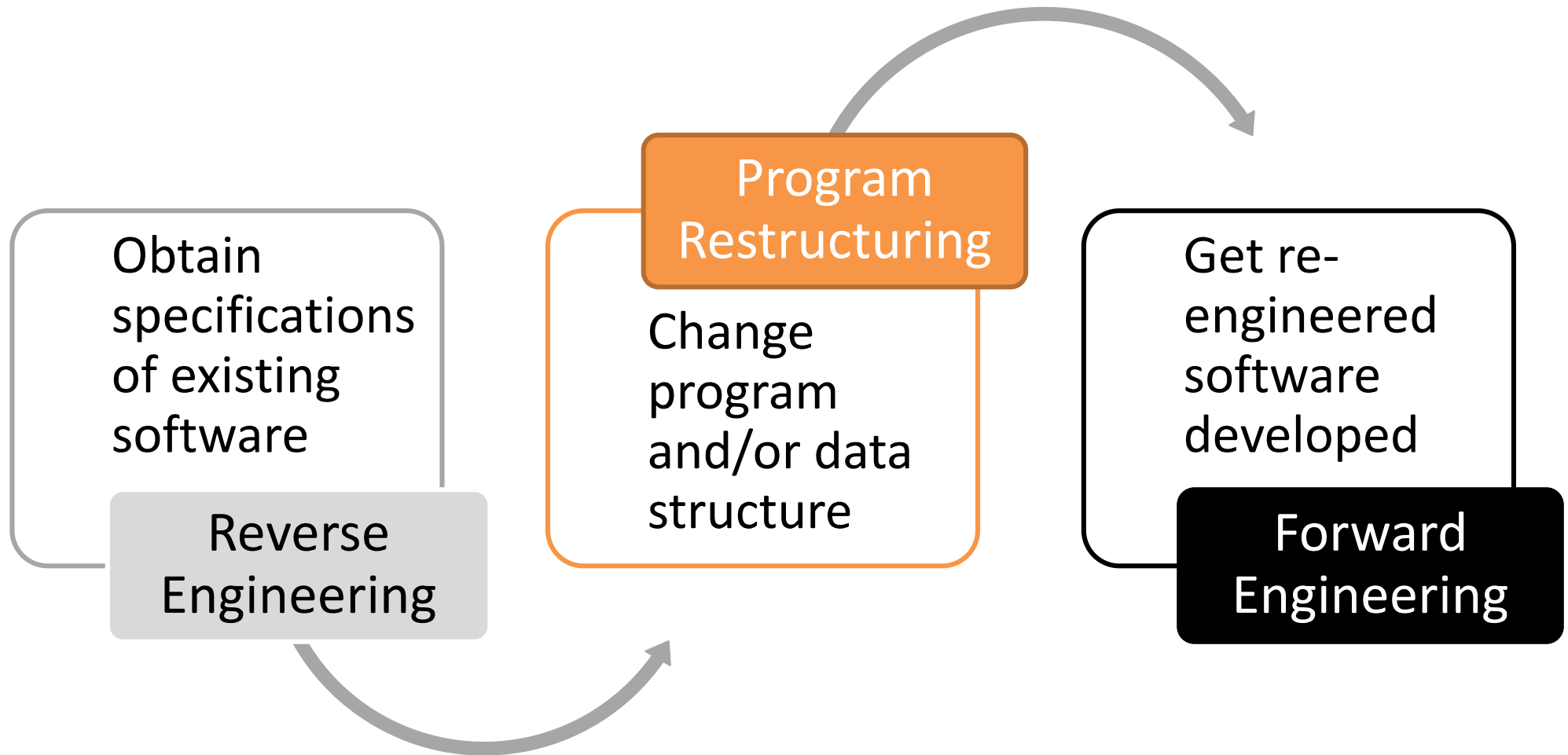
- When we **need** to **update the software** to keep it to the current market, **without impacting** its **functionality**, it is called **software re-engineering**
- It is a **process** where the **design** of software is **changed** and **programs** are **re-written**
- **Legacy software** cannot keep **tuning** with the latest technology available in the market
 - For example, **initially UNIX** was developed in **assembly language**. **When** language **C came** into existence, **UNIX** was **re-engineered** in **C**, because working in assembly language was difficult.
- Other than this, **sometimes programmers notice** that few **parts** of **software** need **more maintenance** than others and they also **need re-engineering**



Re-Engineering Cont.



Re-Engineering Cont.



Re-Engineering Process

- **Decide what to re-engineer.**
 - Is it **whole software** or a **part of it**?
- Perform **Reverse Engineering**, in order **to obtain specifications** of existing **software**
- **Restructure Program** if **required**
 - For example, **changing function-oriented programs** into **object-oriented programs** and **re-structure data** as **required**
- Apply **Forward engineering** concepts in order to **get re-engineered software**



Reverse Engineering

- **Reverse engineering** can **extract design information** from **source code**
- The **abstraction level** of a **reverse engineering process** refers to the **sophistication of the design information** that can be extracted from source code
- Ideally, the **abstraction level** should be **as high as possible**
- The **reverse engineering** process should be **capable of**
 - Deriving **procedural design representations** (a low-level abstraction)
 - **Program** and **data structure information** (a somewhat higher level of abstraction)
 - **Object models, data flow models** (a relatively high level of abstraction)
 - **Entity relationship models** (a high level of abstraction).



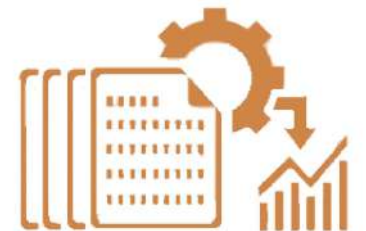
Reverse Engineering Cont.

- As the **abstraction level increases**, information will allow **easier understanding of the program**
- **Interactivity** refers to the **degree** to which the **human** is “**integrated**” with **automated tools** to **create an effective reverse engineering process**
- In most cases, as **the abstraction level increases**, **interactivity must increase**
- The **directionality** of the reverse engineering process is **one-way**, all **information extracted** from the source code is **provided to** the **software engineer**



Forward engineering

- **Forward engineering** is a process of **obtaining desired software from the specifications**, which were brought by reverse engineering
- Forward engineering is **same as software engineering process** with only one difference **it is carried out always after reverse engineering**
- In most cases, **forward engineering** does **not simply create a modern equivalent** of an **older program**
- **Rather**, new **user** and **technology requirements** are **integrated** into the reengineering effort
- The **redeveloped** program **extends** the **capabilities** of the **older application**



Software Configuration Management

The SCM (Software Configuration Management) is a set of **activities** that have been developed **to manage change** throughout the **software life cycle**

Purpose:

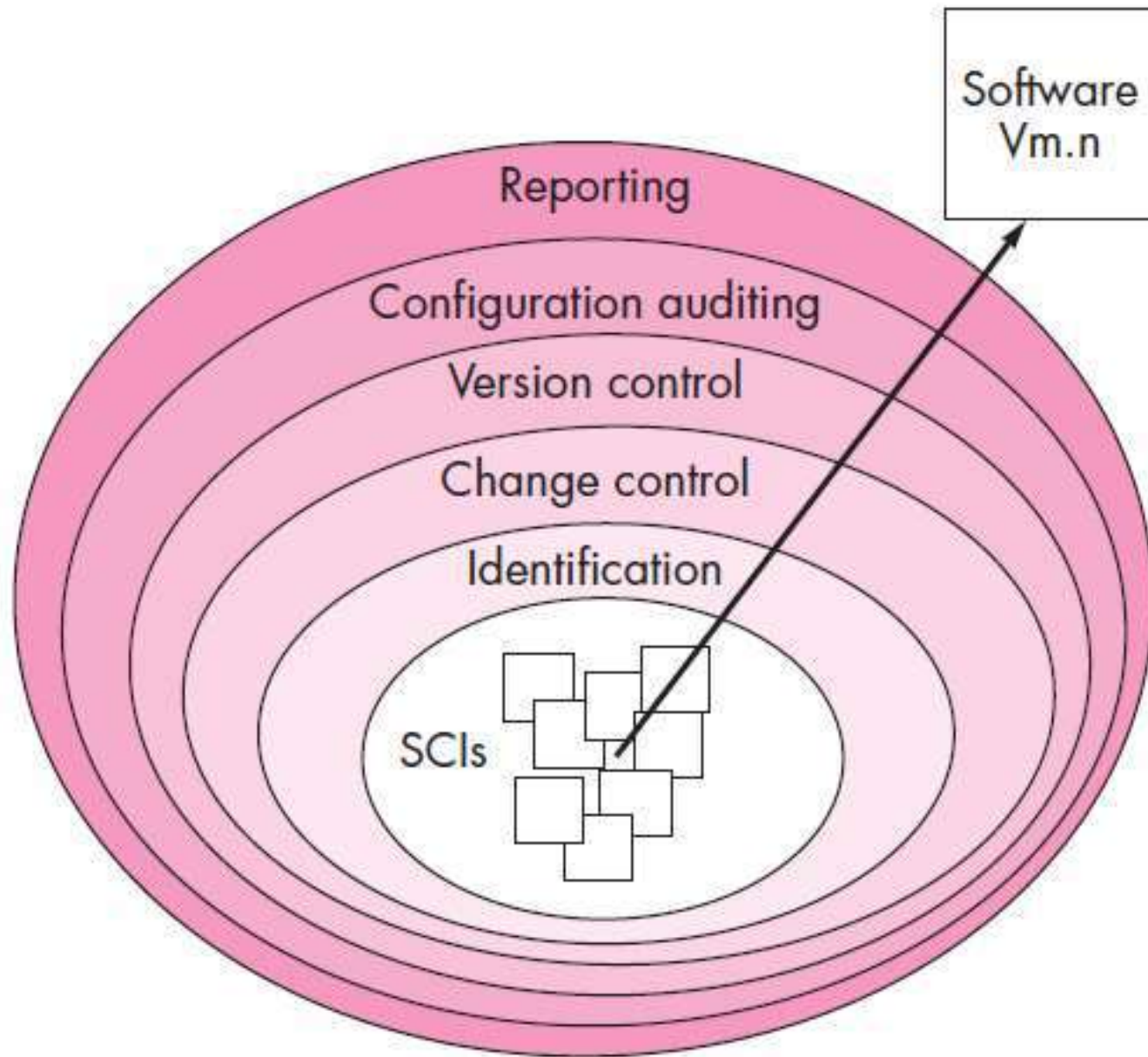
Systematically control changes to the configuration and **maintain** the **integrity** and **traceability** of the configuration throughout the system's life cycle

Four primary objectives

- To **identify** all **items** that collectively **define** the **software configuration**
- To **manage changes** to one or more of these items
- To facilitate the **construction** of different **versions** of an application
- To **ensure** that software **quality** is maintained as the **configuration evolves over time**



Layers of SCM Process



The SCM Process

- Referring to the figure, SCM tasks can be viewed as concentric layers
- **SCIs (Software Configuration Item)** flow outward through these layers throughout their useful life
- As an **SCI moves through a layer**, the **actions** implied by each SCM task **may or may not be applicable**
 - For example, when a new SCI is created, it must be identified.
 - However, if **no changes are requested** for the SCI, the **change control layer does not apply**
- The **SCI is assigned to a specific version** of the software (version control mechanisms come into play)
- A **record** of the **SCI** (its name, creation date, version, etc.) is **maintained** for configuration **auditing purposes**



Identification Task

- **Identification** separately **names** each **SCI** and then **organizes it** in the **SCM repository** using an object-oriented approach
- Objects start out as basic objects and are then grouped into aggregate objects.
- **Each object** has a set of distinct features that identify it.
 - A **name** that is **unambiguous** to all **other objects**
 - A **description** that contains **the SCI type**, a project **identifier**, and **change** and/or **version** information
 - List of **resources needed** by the object
 - The **object realization** (i.e., the document, the file, the model, etc.)



Change Control

- **Change control** is a procedural **activity** that **ensures quality** and **consistency** as changes are made to a configuration object
- A **change request** is submitted to a configuration control authority, which is usually a change control board (**CCB**).
 - The **request** is **evaluated** for **technical merit**, **potential side effects**, **overall impact** on other configuration objects and system functions, and **projected cost** in terms of **money**, **time** and **resources**
- An **engineering change order** (ECO) is **issued** for each approved change request
 - **Describes** the **change** to be made, the **constraints** to follow and the **criteria for review** and **audit**



Change Control Cont.

- The **Base lined SCI** is **obtained** from the SCM **repository**
 - **Access control** governs **which** software **engineers** have the **authority** to **access** and **modify** a particular configuration **object**
 - **Synchronization control** helps to ensure that **parallel changes** performed by two **different people don't overwrite** one another.



Version Control

- **Version control** is a set of **procedures** and **tools** for **managing** the **creation** and **use of multiple occurrences** of **objects** in the **SCM repository**
- **Version Control Capabilities**
 - An SCM **repository** that **stores** all relevant configuration objects
 - A **version management capability** that stores **all versions** of a configuration **object**
 - A make **facility** that **enables** the software **engineer** to **collect** all **relevant configuration objects** and **construct a specific version** of the **software**
 - **Issues or bug tracking** capability that **enables** the **team** to **record** and **track** the **status of all outstanding issues** associated with **each** configuration **object**



Version Control cont.

- The **SCM repository** maintains a **change set**
 - Serves as a **collection of all changes** made to a baseline configuration
 - Used to **create a specific version** of the **software**
 - **Captures** all **changes** to all **files** in the configuration along **with the reason** for changes and details of **who made the changes** and **when**

Few version control systems



Configuration Audit

- **Configuration auditing** is an **SQA activity** that **helps** to ensure that **quality** is **maintained** as **changes** are made.
- It complements the **formal technical review** and is **conducted** by the **SQA group**
- It **addresses** the following **questions**
 - Has a **formal technical review** been **conducted** to assess technical **correctness**?
 - Has the **software process been followed** and have **software engineering standards** been properly **applied**?
 - Has the **change** been "**highlighted**" and "**documented**" in the **SCI**? Have the **change data** and **change author** been **specified**? Do the attributes of the configuration object reflect the change?



Configuration Audit Cont.

- Have **SCM procedures** for noting the change, recording it and reporting it been **followed**?
- Have all related **SCIs** been properly **updated**?
- A configuration audit ensures that
 - The **correct SCIs** (by version) have been **incorporated** into a **specific build**
 - That **all documentation** is **up-to-date** and **consistent with** the **version** that has been **built**



Status Reporting

- **Configuration status reporting (CSR)** is also called **status accounting**
- Provides **information about** each **change** to those **personnel** in an **organization** with a **need to know**
- **Answers** what happened, who did it, when did it happen and what else will be affected?
- **Sources of entries** for configuration status reporting
 - Each time a **SCI** is assigned new or updated information
 - Each time a **configuration audit** is conducted
- **The configuration status report**
 - **Placed** in an **on-line database** or on a **website** for **software developers** and **maintainers** to read
 - **Given** to **management** and **practitioners** to keep them apprised of important changes to the project SCIs

CO5:

1) SQA

2) FTR

3) SQA & SQC (Difference)

4) Quality Standards

5) SaaS (optional)

6) CASE (optional)

What is Quality

Quality

Developed product meets it's specification

Quality Management

Ensuring that required level of product quality is achieved

Defining procedures and standards

Applying procedures and standards to the product and process

Checking that procedures are followed

Collecting and analyzing various quality data

Software Quality Assurance (SQA)

- ▶ **Software quality assurance** (also called **quality management**) is an **umbrella activity** that is applied throughout the software process
- ▶ It is planned and systematic pattern of **activities** necessary to **provide high degree of confidence in the quality**
- ▶ Software quality assurance (**SQA**) encompasses
 - An **SQA process**
 - Specific **quality assurance** and **quality control tasks**
 - **Effective software engineering** practice
 - **Control** of all software **work products**
 - A **procedure** to **ensure compliance** with software **development standards**
 - **Measurement** and **reporting mechanisms**

SQA Activities

- ▶ **Prepare** an **SQA plan** for a project
- ▶ **Participate** in the **development** of the project's **software process** description
- ▶ **Review** software engineering **activities** to **verify compliance** with the **defined software process**
- ▶ **Audit** designated **software work products** to verify compliance with those defined as part of the software process
- ▶ **Ensure** that **deviations** in software **work** and **work products** are **documented** and **handled** according to a documented procedure
- ▶ **Records** any **noncompliance** and reporting **to senior management**

SQA Techniques

- ▶ Statistical **quality assurance** implies **the following steps**:
 1. **Information** about software **defects is collected** and **categorized**
 2. **An attempt** is made to **trace each defect** to its underlying **cause**
 - Ex., non-conformance to specifications, design error, violation of standards, poor communication with the customer
 3. **Using** the **Pareto principle** (80 percent of the defects can be traced to 20 percent of all possible causes), **isolate the 20 percent** (the "vital few").
 4. Once the vital few causes have been identified, **move to correct the problems** that have caused the defects.
- ▶ Some of the defects are uncovered as software is being developed.
- ▶ Other are encountered after the software has been released.

SQA Techniques (Cont.)



Uncovered **errors can be tracked to** (one or more) of the **following causes**

- ▶ Incomplete or erroneous specifications (IES)
- ▶ Misinterpretation of customer communication (MCC)
- ▶ Intentional deviation from specifications (IDS)
- ▶ Violation of programming standards (VPS)
- ▶ Error in data representation (EDR)
- ▶ Inconsistent component interface (ICI)
- ▶ Error in design logic (EDL)
- ▶ Incomplete or erroneous testing (IET)
- ▶ Inaccurate or incomplete documentation (IID)
- ▶ Error in programming language translation of design (PLT)
- ▶ Ambiguous or inconsistent human/computer interface (HCI)
- ▶ Miscellaneous (MIS)

Software Reviews (Formal Technical Reviews)

- ▶ A formal technical **review (FTR)** is a software **quality control activity** performed by software engineers (and others)
- ▶ **The objectives of an FTR are:**
 1. To **uncover errors** in **function**, **logic**, or **implementation**; for any representation of the software
 2. To **verify** that the **software under review meets** its **requirements**
 3. To **ensure** that the software has been **represented according to predefined standards**
 4. To **achieve software** that is **developed** in a **uniform manner**
 5. To **make projects** more **manageable**
- ▶ During the **FTR**,
 - ↳ a **reviewer** (the recorder) actively **records** all **issues** that have been raised
- ▶ These are **summarized at the end of** the review **meeting**, and a reviewed issues list is produced
- ▶ In addition, a formal technical **review summary report** is **completed**

Review Guidelines

Guidelines for conducting formal technical reviews **must be** established in **advance**, **distributed** to all **reviewers**, **agreed** upon & then **followed**

Review the **product**, **not** the **producer**

Set an **agenda** and **maintain** it

Limit debate and denial

Speak problem areas, but don't attempt to solve every problem noted

Take written **notes**

Conduct meaningful **training** for all **reviewers**

Limit the number of **participants** and insist upon advance preparation

Develop a **checklist** for each product that is likely to be reviewed

Allocate resources and **schedule** time for **FTRs**

Review your **early reviews**

Differentiation of SQA & SQC



Criteria	SQA	SQC
Definition	SQA is a set of activities for ensuring quality in software engineering processes (that ultimately result in quality in software products). The activities establish and evaluate the processes that produce products	SQC is a set of activities for ensuring quality in software products . The activities focus on identifying defects in the actual products produced
Focus	Process focused	Product focused
Orientation	Prevention oriented	Detection oriented
Breadth	Organization wide	Product/project specific
Scope	Relates to all products that will ever be created by a process	Relates to specific product
Activities	Process Definition and Implementation, Audits, Training	Reviews, Testing

The quality standards

ISO 9000 and 9001
Six Sigma
CMM



ISO 9001

- ▶ In order to **bring quality** in **product** and **service**, many organizations are **adopting Quality Assurance System**
- ▶ **ISO standards** are **issued by** the **International Organization for Standardization (ISO)** in **Switzerland**
- ▶ **Proper documentation** is **an important part** of an ISO 9001 Quality Management System.
- ▶ **ISO 9001** is the **quality assurance standard** that **applies** to **software engineering**
- ▶ It **includes**, **requirements** that **must be present** for an effective quality assurance system
- ▶ ISO 9001 standard is **applicable to all engineering discipline**



ISO 9001 Cont.

The **requirements described by ISO 9001:2000** address topics such as

Management Responsibility

Quality System

Design Control

Document

Product Identification

Traceability

Inspection

Testing

Control of Quality Records

Internal Quality

Training

Servicing

Contract Review

Process Control

Data Control

Preventive Action

Audits

In order for a **software organization** to become **registered to ISO 9001:2000**

1. It must **establish policies** and **procedures** to **address** each of the **requirements** just **noted**
2. **Able to demonstrate** that these **policies** and **procedures** are being **followed**



Six Sigma

Several Software Packages available to assist in measuring yield, defects per million opportunities, etc.



- ▶ Six sigma is “**A generic quantitative approach to improvement that applies to any process**”
- ▶ **Six Sigma** is a disciplined, **data-driven approach** and **methodology** for **eliminating defects** in **any process** - from manufacturing to transactional and from product to service
- ▶ To **achieve six sigma**, a **process** must **not produce more than 3.4 defects per million opportunities**
 - ➔ **4 Sigma** → **6210** defects per million opportunities
 - ➔ **5 Sigma** → **230** defects per million opportunities
- ▶ Six sigma have two methodologies.
 - ➔ **DMAIC** (Define, Measure, Analyze, Improve, Control)
 - ➔ **DMADV** (Define, Measure, Analyze, Design, Verify)

DMAIC - Six Sigma

Define: Define the problem or process to improve upon related to the customer and goals

Measure: How can you measure this process in a systematic way?

Analyze: Analyze the process or problem and identify the way in which it can be improved. What are the root causes of problems within the process?

Improve: Once you know the causes of the problems, present solutions for them and implement them.

Control: Utilize Statistical Process Control to continuously measure your results and ensure you are improving

DMADV - Six Sigma

Define, Measure and **analyze** are similar to above method

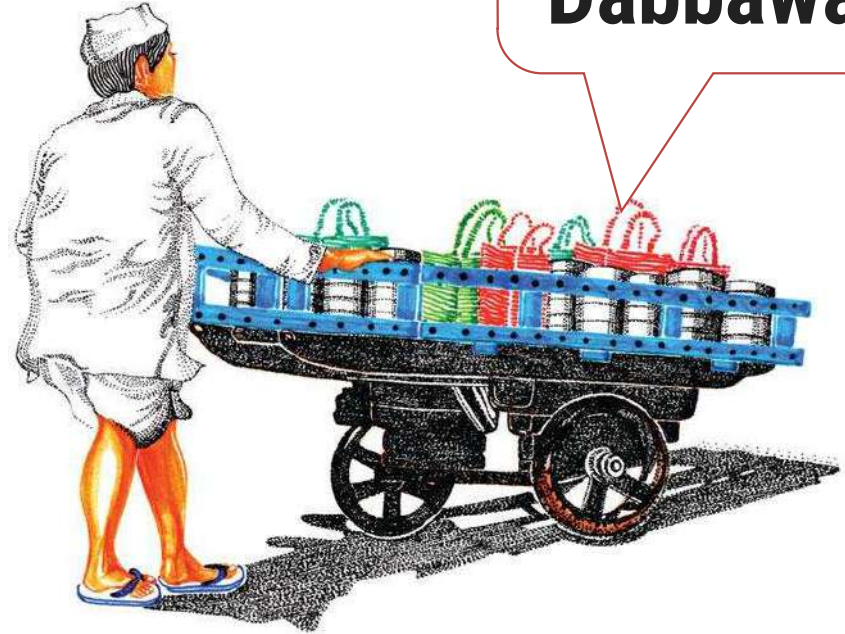
Design: **Avoid root causes** of **defects** and **meet** the customer **requirements**

Verify: To **verify** the **process**, **compare** the **process** with the **standard plan** and **find differences**



Example of Six Sigma Company

Mumbai's Dabbawalas



For **over 100 years** they have **delivered food** to **every part of the city**, earning them a **Six Sigma rating** (a Forbes rating of 99.9 % which **means one error in 6 million transactions**).

CMM (Capability Maturity Model)

- ▶ To **determine** an **organization's current state** of **process maturity**, the **SEI (Software Engineering Institute)** uses an assessment that results in a **five point grading scheme**
- ▶ The grading scheme determines compliance with a **capability maturity model (CMM)** that **defines key activities required** at **different levels** of **process maturity**

The **SEI** approach establishes **five process maturity levels** that are defined in the following manner

Level 1: Initial

- The software **process** is **characterized** as **ad hoc** and **occasionally**
- Few processes are defined and **success depends** on **individual effort**

Level 2: Repeatable

- Basic project management **processes** are **established** to track cost, schedule, and functionality.
- The necessary **process discipline** is in place to **repeat earlier successes** on Project

Level 3: Defined

- The software **process** for **both management** and **engineering** activities is **documented, standardized** and **integrated**
- This level **includes** all **characteristics** defined for **level 2**

CMM (Capability Maturity Model) Cont.

Level 4: Managed

- Detailed **measures** of the software **process** and **product quality** are **collected**
- This level **includes** all **characteristics** defined for **level 3**

Level 5: Optimizing

- Continuous **process improvement** is **enabled** by quantitative **feedback** from the **process** and **from testing innovative ideas** and **technologies**
- This level **includes** all **characteristics** defined for **level 4**



SQA Plan



- ▶ The **SQA Plan** provides a **road map** for **establishing** software **quality assurance**
- ▶ The plan **serves as a template** for SQA activities that are instituted for each software project
- ▶ The standard recommends a structure that identifies:
 - ↳ The **purpose** and **scope** of the **plan**
 - ↳ A **description** of **all** software engineering **work products** (e.g., models, documents, source code).
 - ↳ **All** applicable **standards** that are **applied during** the **software process**
 - ↳ **SQA actions** and **their placement** throughout the software process
 - ↳ The **tools** and **methods** that **support** SQA **actions** and **tasks**
 - ↳ Software **configuration management procedures**
 - ↳ **Methods** for **assembling, safeguarding** and **maintaining** all **SQA-related records**
 - ↳ Organizational **roles** and **responsibilities** relative to product **quality**

Software as a Service

- **SaaS (Software as a Service)** describes any **cloud service** where **consumers** are able to **access software applications** over the **internet**
 - **Google, Twitter and Facebook** are all **examples of SaaS**, with users able to access the services via any internet enabled device
 - **Enterprise users** are able to use applications for a range of needs, including **accounting** and **invoicing, tracking sales, planning, performance monitoring** and **communications**
- SaaS is often referred to as **software-on-demand** and utilizing it as **renting software** rather than buying it
- With traditional software applications you would purchase the software upfront as a package and then install it onto your computer

Software as a Service cont.

- The **software's license** may also **limit the number of users** and/or **devices where the software can be deployed**
- **Software as a Service** users, however, **subscribe** to the software rather than purchase it, **usually on a monthly basis**
- **Applications are purchased** and **used online** with files (**data files**) **saved in the cloud** rather than on individual computers

Software as a Service Cont.

Benefits of **SaaS** to organizations and personal users

No additional hardware costs

Updates are automated

No initial setup costs

Accessible from any location

Pay for what you use

Cross device compatibility

Usage is scalable

Applications can be customized

Software as a Service Cont.

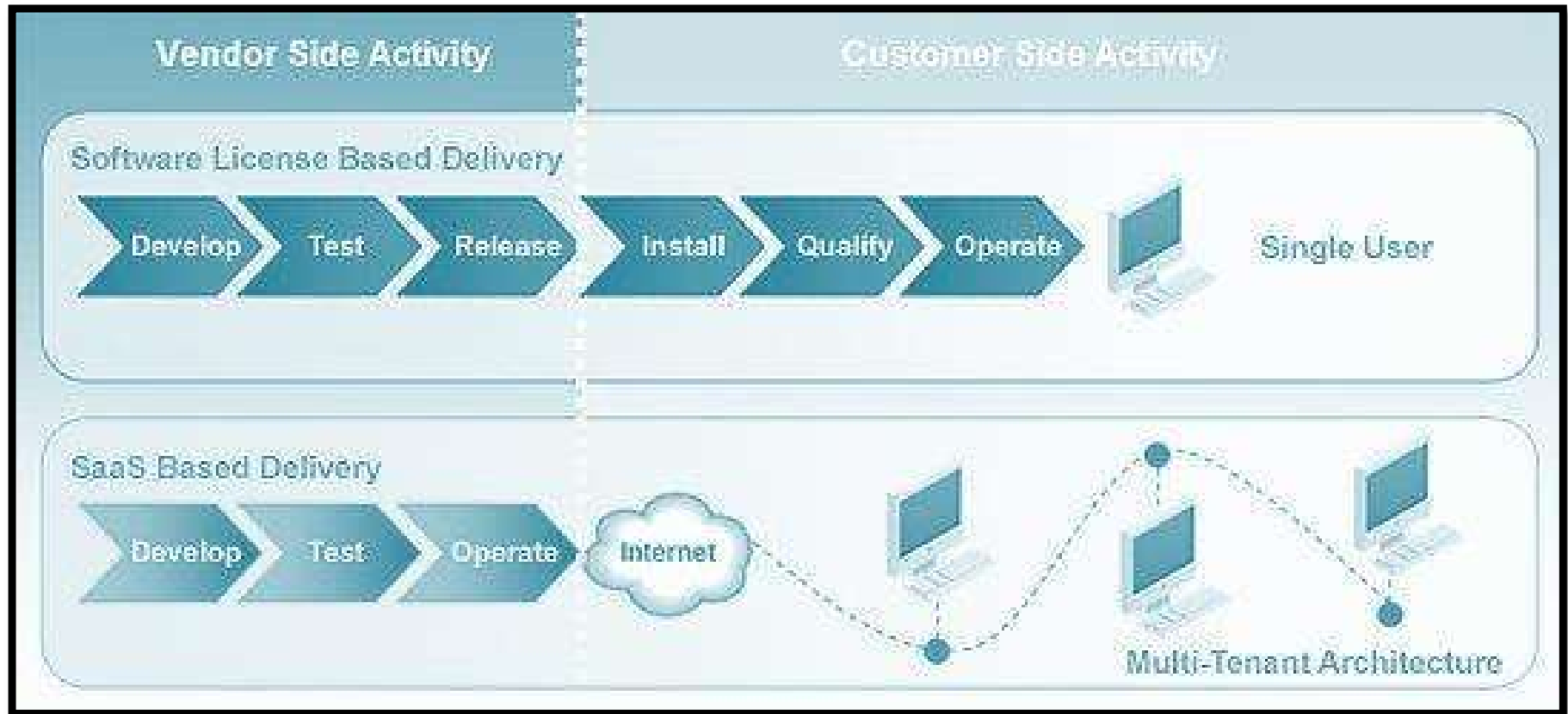
- **Office software** is the best **example** of **businesses utilizing SaaS**
- Tasks related to accounting, invoicing, sales and planning can all be performed through Software as a Service.
- The required software can be subscribed via the internet and then accessed online via any computer in the office using a username and password.
- If needs change they can easily switch to software that better meets their requirements.
- Everyone who needs access to a particular piece of software can be set up as a user
- Whether it is one or two people or every employee in a corporation.

SaaS Summary

- There **are no setup costs** with SaaS, as there often are with other applications
- SaaS **is scalable with upgrades** available **on demand**
- Access to Software as a Service is **compatible across all internet enabled devices**
- As long as there is an internet connection, **applications are accessible from any location**

SaaS Architecture

Traditional Software Delivery Model



SaaS Model

SaaS Architecture (Cont...)

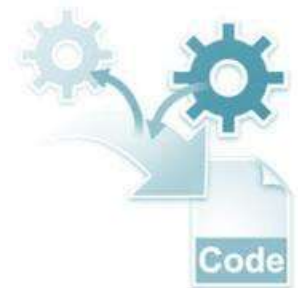
- The diagram illustrates the key **differences** between the **traditional software delivery** model and a **SaaS-based** delivery model
 - In essence, **all users of SaaS-based** applications **run exactly the same code** with customizations and configurations stored as metadata parameters
- The **key difference** between **SaaS** and **traditional software delivery** is that **SaaS-based applications** are **fully hosted within some form of Data Centre environment**
 - All updates to the SaaS application are carried out within the Data Centre and therefore the **user** can always be assured that they **are using the most recent version of the application**
 - These **applications** are designed to work and take advantage of **operating within a web browser-based environment**

SaaS Architecture (Cont...)

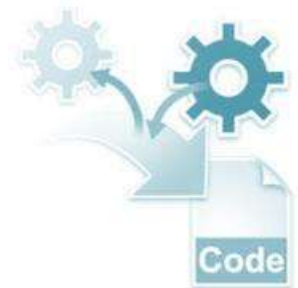
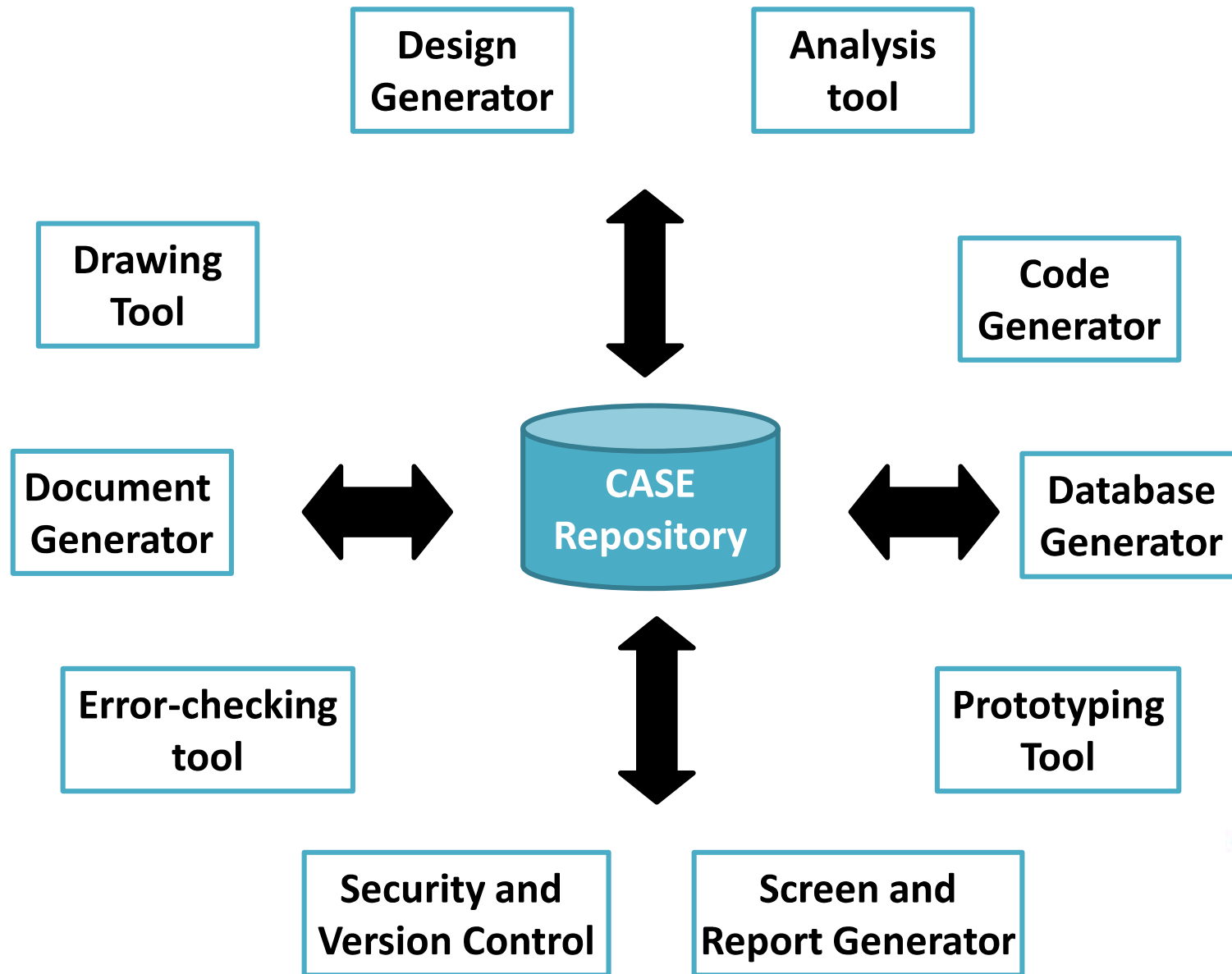
- Since nearly every **computer comes with a browser** as part of the operating environment, there is **hardly any implementation work required by a company's IT department**
- This allows the IT department to focus on other activities **reducing support costs**
- **SaaS** is **traditionally sold** on a **subscription basis** that **includes upgrades, maintenance** and some level of **basic support**
- These subscriptions operate on a monthly, quarterly or yearly basis
- SaaS is **ideal for companies** who would be **using the application periodically**

Computer-Aided Software Engineering

- A **CASE (Computer Aided Software Engineering)** tool is a generic term used to denote **any form of automated support** for **software engineering**
- A **CASE tool** means any tool used to **automate some activity** associated with **software development**
- The primary **reasons** for using a **CASE tool** are:
 - To **increase productivity**
 - To **help produce better quality software** at lower cost



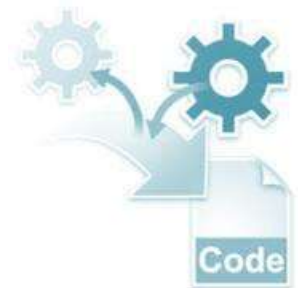
Components of CASE



Components of CASE Cont.

■ CASE repository

- **Central component** of any CASE tool
- Also known as the **information repository** or **data dictionary**
- **Centralized database**
- Allows **easy sharing of information** between tools and SDLC activities
- Used to **store graphical diagrams** and **prototype forms** and **reports** during analysis and design workflows
- Provides **wealth of information** to **project manager** and allows control over project
- Facilitates **reusability**



Components of CASE Cont.

■ Diagramming tools

- Allow you **to represent a system and its components visually**
- Allows **higher level processes to be easily decomposed**
- Can examine processes or data models at high or low level

■ Screen and report generators

- Used **to Create, modify and test prototypes** of computer **displays and reports**
- **Identify** which data **items to display or collect for each screen or report**
- Some tools have templates



Components of CASE Cont.

■ Analysis tools

- Generate **reports** that help **identify possible inconsistencies, redundancies** and **omissions** (deficiency)
- Generally focus on
 - **diagram completeness and consistency**
 - **data structures** and **usage**

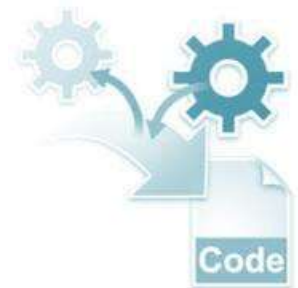
■ CASE documentation generator tools

- **Create standard reports** based on **contents of repository**
- Need **textual descriptions** of requirements, solutions, diagrams of data and processes, prototype forms and reports, program specifications and user documentation
- **High-quality documentation leads to 80% reduction in system maintenance effort** in comparison to average quality documentation



CASE Tool Classification

- **Business process engineering tools**
 - represent business data objects, their relationships and flow of the data objects between company business areas
- **Process modeling and management tools**
 - represent key elements of processes and provide links to other tools that provide support to defined process activities
- **Project planning tools**
 - used for cost and effort estimation and project scheduling
- **Risk analysis tools**
 - help project managers build risk tables by providing detailed guidance in the identification and analysis of risks



CASE Tool Classification Cont.

- **Requirements tracing tools**

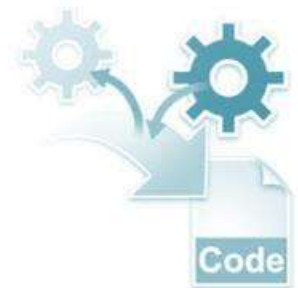
- provide systematic database-like approach to tracking requirement status beginning with specification

- **Metrics and management tools**

- management oriented tools capture project specific metrics that provide an overall indication of productivity or quality

- **Documentation tools**

- provide opportunities for improved productivity by reducing the amount of time needed to produce work products



CASE Tool Classification Cont.

- **System software tools**

- network system software, object management services, distributed component support and communications software

- **Quality assurance tools**

- metrics tools that audit source code to determine compliance with language standards or tools that extract metrics to project the quality of software

- **Database management tools**

- RDBMS and OODMS (object-oriented database management system) serve as the foundation for the establishment of the CASE repository

